



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА



Кафедра цифровой трансформации

Институт информационных технологий

Презентации по лекционным материалам по дисциплине «Разработка баз данных»

Направления подготовки:

Уровень: бакалавриат

Форма обучения: очная

01.03.04 Прикладная математика
09.03.03 Прикладная информатика
09.03.04 Программная инженерия
09.03.01 Информатика и вычислительная техника

**Лекция №8 Типы данных JSON/JSONB и BYTEA. Управление правами доступа.
Масштабирование баз данных.**

Лектор
Исаева Мария Владимировна
Кандидат технических наук,
доцент кафедры
цифровой трансформации

2025/2026 учебный год

СВЕДЕНИЯ О ДИСЦИПЛИНЕ

Лекции ведут:

- Исаева Мария Владимировна, к.т.н., доцент каф. Цифровой трансформации
- Дзгоев Алан Эдуардович, к.т.н., доцент каф. Цифровой трансформации
(Dzgoviev@mirea.ru / *При обращении в теме письма указывайте номер группы*)
- Миронов Антон Николаевич, Старший преподаватель каф. Цифровой трансформации
- Семькина Наталья Александровна, д.т.н., профессор каф. Цифровой трансформации
- Резеньков Роман Николаевич, к.т.н., доцент каф. Цифровой трансформации

Объём **аудиторной** работы по дисциплине в 5 семестре:

- Лекции – 16 часов;
- Практические занятия – 48 часа;
- Аттестация – экзамен.

Допуск к экзамену:

- посещение лекций и практических работ;
- выполнение в установленный срок всех практических работ.

подробности в памятке по дисциплине (следующие слайды)

Выполнение практических заданий

1. Все материалы курса на <https://online-edu.mirea.ru> в разделе **Разработка баз данных: доступ к материалам курса открывается по расписанию занятий.**

2. Доступ с аккаунта МИРЭА.

3. Все создаваемые файлы прикладываются к заданию.

4. Учёт выполненных работ.

5. Общение с преподавателем через отзывы в заданиях.

Курс	Л4_ % кликов по кнопке "контроль присутствия"	Л5_ % кликов по кнопке "контроль присутствия"	Л6_04-05_ % присутствия	Л7_18-05_ % присутствия	Л8_01-06_ % присутствия	%СРЗНАЧ участия в лекциях	Задание:Практика 1_Законодательная и нормативно-техническая база стандартизации в РФ	Задание:Практика 2_Нормативные документы по стандартизации ИТ	Задание:Практика 3_ч1_Классификаторы в области ИТ	Задание:Практика 3_ч2_Классификаторы в области ИТ	Задание:Практика 3_ч3_Классификаторы в области ИТ	Задание:Практика 4_ч1_Создание технического задания в соответствии с ГОСТ 34.602-2020
0	0	50				6,3	-	-	-	-	-	-
0	0	50		100	100	31,3	-	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
00	0	50	100		100	56,3	1	1	1	1	1	1
100	100	100	100	100	100	87,5	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	50	0				6,3	1	-	-	-	-	-
100	0	100		100		56,3	1	1	1	1	1	1
0	0	0				12,5	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	-	-	-	-	-	-
0	100	100	100	100	100	68,8	1	1	1	1	1	1
0	0	50				18,8	-	-	-	-	-	-
0	0	0	100	100	100	50	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
0	100	100	100	100	100	100	1	1	1	1	1	1
0	0	0				0	-	-	-	-	-	-
100	0	0				18,8	1	1	1	1	-	-
50	0	50				18,8	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0				0	-	-	-	-	-	-
0	0	0			100	12,5	1	1	1	1	1	1
100	100	0				25	1	1	-	-	-	-
0	0	0				0	-	-	-	-	-	-
100	100	100	100	100	100	100	1	1	1	1	1	1

Рекомендуемая литература

1. Новиков Б.А. Основы технологий баз данных: учебное пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под.ред. Е.В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. Режим доступа: <https://postgrespro.ru/education/books/dbtech>
2. Моргунов Е.П. PostgreSQL. Основы языка SQL: учеб. Пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил. Режим доступа: <https://postgrespro.ru/education/books/sqlprimer>
3. Комаров В.И. Путеводитель по базам данных. – М.: ДМК-Пресс, 2024. – 520 с. Режим доступа: <https://edu.postgrespro.ru/dbguide.pdf>
4. Смирнов М.В. Проектирование баз данных [Электронный ресурс]: Конспект лекций / Смирнов М.В. - М., МИРЭА – Российский технологический университет, 2020 – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/163892/>
5. Чистякова М.А. Проектирование и эксплуатация баз данных [Электронный ресурс]: Учебно-методическое пособие / Чистякова М.А., Иванова И.А., Котилевец И.Д. – М.: МИРЭА – Российский технологический университет, 2021. – 1 электрон. Опт. диск (CD-ROM). Режим доступа: <https://e.lanbook.com/book/176572/>
6. Братусь Н.В. Базы данных. Часть 1 [Электронный ресурс]: Практикум / Братусь Н.В., Маличенко С.В., Матчин В.Т. – М.: МИРЭА – Российский технологический университет, 2024. – 1 электрон. опт. диск (CD-ROM) – Режим доступа: <https://ibc.mirea.ru/books/SHARE/5859/>
7. Моргунов Е. П. PostgreSQL. Профессиональный SQL : учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова. – М.: ДМК Пресс, 2025. – 444 с. Режим доступа: <https://postgrespro.ru/education/books/advancedsql>
8. Рогов Е. В. PostgreSQL 17 изнутри. – М.: ДМК Пресс, 2025. – 668 с. Режим доступа: <https://postgrespro.ru/education/books/internals>
9. Введение в системы баз данных. : Пер. с англ. / К. Дж. Дейт .— М. : Изд. дом "Вильямс", 2001 .— 1072 с. https://ibc.mirea.ru/books/search/?search_field=Дейт&page=3

Неделя	Лекции	Практические работы	Дедлайны, текущие и контрольные мероприятия
1	Лекция 1. Тема: Реляционная модель данных и базовые операции SQL. (2 часа)	Занятие 1. ПРАКТИЧЕСКАЯ РАБОТА №1. Создание БД и запросы на выборку данных (2 часа)	
2		Занятие 2. ПРАКТИЧЕСКАЯ РАБОТА №1. (2 часа)	Дедлайн по защите отчета по практике №1.
		Занятие 3. ПРАКТИЧЕСКАЯ РАБОТА №2. Многотабличные запросы, использование операций объединения, пересечения, разности (2 часа)	
3	Лекция 2. Тема: Многотабличные запросы и теоретико-множественные операции. (2 часа)	Занятие 4. ПРАКТИЧЕСКАЯ РАБОТА №2. (2 часа)	
4		Занятие 5. ПРАКТИЧЕСКАЯ РАБОТА №2. (2 часа)	Дедлайн по защите отчета по практике №2.
		Занятие 6 ПРАКТИЧЕСКАЯ РАБОТА №3 Использование подзапросов и CTE (2 часа)	
5	Лекция 3. Использование подзапросов и агрегатных выражений (2 часа)	Занятие 7. ПРАКТИЧЕСКАЯ РАБОТА №3 (2 часа)	Тест №1. (по лекциям №1-2).
6		Занятие 8. ПРАКТИЧЕСКАЯ РАБОТА №3 (2 часа).	Дедлайн по защите отчета по практике №3.
		Занятие 9. ПРАКТИЧЕСКАЯ РАБОТА №4 Использование оконных функций и функций ранжирования (2 часа)	
7	Лекция 4. Тема: SQL. Оконные функции и аналитические запросы (2 часа)	Занятие 10. ПРАКТИЧЕСКАЯ РАБОТА №4 (2 часа)	Дедлайн по защите отчета по практике №4.
8		Занятие 11. ПРАКТИЧЕСКАЯ РАБОТА №4 (2 часа)	Дедлайн по защите отчета по практике №4.
		Занятие 12. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	
9	Лекция 5. Операторы модификации данных, объекты базы данных (2 часа)	Занятие 13. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	Дедлайн по защите отчета по практике №5.

План – график лекций и практических занятий

10		Занятие 14. ПРАКТИЧЕСКАЯ РАБОТА №5 (2 часа)	Дедлайн по защите отчета по практике №5.
		Занятие 15. ПРАКТИЧЕСКАЯ РАБОТА №6 (2 часа)	
11	Лекция 6. Управление данными: триггеры, курсоры (2 часа)	Занятие 16. 2 часа	Контрольная работа №1.
12		Занятие 17. ПРАКТИЧЕСКАЯ РАБОТА №6 (2 часа)	Дедлайн по защите отчета по практике №6.
		Занятие 18. ПРАКТИЧЕСКАЯ РАБОТА №7 Оптимизация запросов (2 часа)	Тест №2. (по лекциям №3-5).
13	Лекция 7. Оптимизация запросов (2 часа)	Занятие 19. ПРАКТИЧЕСКАЯ РАБОТА №7 (2 часа)	
14		Занятие 20. ПРАКТИЧЕСКАЯ РАБОТА №7 (2 часа)	Дедлайн по защите отчета по практике №7.
		Занятие 21. ПРАКТИЧЕСКАЯ РАБОТА №8 Хранение неструктурированных данных (2 часа)	
15	Лекция 8. Администрирование баз данных и хранение неструктурированных данных (2 часа) Тест №3. (по лекциям №6-7).	Занятие 22. ПРАКТИЧЕСКАЯ РАБОТА №8 (2 часа)	
16		Занятие 23. ПРАКТИЧЕСКАЯ РАБОТА №8 (2 часа)	Дедлайн Практики №8.
		Занятие 24. 2 часа	Контрольная работа №2.
Итого	16 часов	48 часов	

ПАМЯТКА

о правилах обучения дисциплины

«Разработка баз данных» (2025-2026, I семестр)

№	Наименование	Формат	Период проведения	Баллы (макс.)
1	Защита практических работ (8 практик)	Очно	Сентябрь 2025 – Декабрь 2025	48 (6 баллов за 1 практику)
2	Контрольный тест №1 (по лекциям №1 и №2)	Очно, СДО	Октябрь 2025	5
3	Контрольный тест №2 (по лекциям №3 - №5)	Очно, СДО	Ноябрь 2025	5
4	Контрольная работа №1	Очно, СДО	Ноябрь 2025	10
5	Контрольный тест №3 (по лекциям №6 - №8)	Очно, СДО	Декабрь 2025	6
6	Контрольная работа №2	Очно, СДО	Декабрь 2025	10
7	Посещение (лекции 8 занятий, практики 24 занятия)	Очно	Сентябрь– Декабрь 2025	16 б. 0,5 баллов за посещения 1 лекции и практики (0,5·32 пар)
Максимальная сумма баллов за активность по дисциплине в течение учебного семестра				100

ДИСЦИПЛИНА	Разработка баз данных (укажите полное наименование дисциплины без сокращений)
ИНСТИТУТ	информационных технологий (укажите название учебного института)
КАФЕДРА	Цифровой трансформации (укажите полное наименование кафедры)
Уровень обучения	Бакалавриат
Аудиторные часы	16/0/48 (укажите количество часов лекционных, лабораторных и практических)

Лекции 16 часов.

Обязательное посещение всех лекций.

В тестах в рамках мероприятий текущего контроля вопросы из лекций

Практические работы (8 работ)

Обязательное посещение всех практических занятий.

Допуск к экзамену – очная защита всех практических работ в течение семестра. Защита отчёта до дедлайна

Защита практической работы после дедлайна – 0 баллов, но работа засчитывается.

Если есть справка по перенесённой болезни, заверенная в учебном отделе, то эту справку необходимо принести и показать преподавателю по практике. В таком случае назначается пересдача пропущенной практической работы в день консультаций (важно: студент защищает именно ту практическую работу, которую он пропустил по болезни, согласно датам в справке). Если студент пропустил практические работы без уважительной причины и, соответственно, работу не защитил, то на экзамен он не допускается.

Если студент загрузил отчет до дедлайна, но не защитил, то такой отчет не засчитывается студенту, т.е. не сдал.

Дополнительного времени на защиту практических работ не выделяется.

Выполненные работы студент загружает в СДО в формате pdf. Фон области построения модели – белый (в отчёте).

№	Наименование	Формат	Период проведения	Баллы (макс.)
1	Защита практических работ (8 практик)	Очно	Сентябрь 2025 – Декабрь 2025	48 (6 баллов за 1 практику)
2	Контрольный тест №1 (по лекциям №1 и №2)	Очно, СДО	Октябрь 2025	5
3	Контрольный тест №2 (по лекциям №3 - №5)	Очно, СДО	Ноябрь 2025	5
4	Контрольная работа №1	Очно, СДО	Ноябрь 2025	10
5	Контрольный тест №3 (по лекциям №6 - №8)	Очно, СДО	Декабрь 2025	6
6	Контрольная работа №2	Очно, СДО	Декабрь 2025	10
7	Посещение (лекции 8 занятий, практики 24 занятия)	Очно	Сентябрь– Декабрь 2025	16 б. 0,5 баллов за посещения 1 лекции и практики (0,5·32 пар)
Максимальная сумма баллов за активность по дисциплине в течение учебного семестра				100

Если студент не загрузил отчет до дедлайна, то загрузка после дедлайна станет недоступна.

Контрольные тесты

Тесты студенты выполняют в СДО на паре.

Баллы минусуются, если по базовым вопросам студент отвечает не правильно.

Проходного балла нет, сколько студент набрал столько и остается.

Контрольный тест №1. (Лекции 1, 2) проводится на 5-м практическом занятии в начале пары (20 минут).

Контрольный тест №2. (Лекции 3, 4, 5) проводится на 18-м практическом занятии в начале пары (20 минут)..

Контрольный тест №3 (Лекции №6-7) проводится на 8-й лекции в начале пары (20 минут).

Контрольные работы

Выполняются на 16 и 24 практических занятиях в аудитории.

Студент получает задание на паре.

Время выполнения контрольной работы – 1 час.

Экзамен

Допуск к экзамену – защита всех практических работ в течение семестра очно. В случае, если студент не сдал какие-либо практики, у него есть возможность сдать их во время экзамена, если успеет. В противном случае, отправляется на пересдачу (необходимость сдачи работ сохраняется).

Если студент набрал меньше 50 % от общей суммы баллов, то экзамен будет проходить по билету (2теоретических вопроса + 1 задача).

Если студент набрал больше 50% от суммы максимальной суммы баллов, то сдаёт тестирование на экзамене. + добавляются баллы за активность.

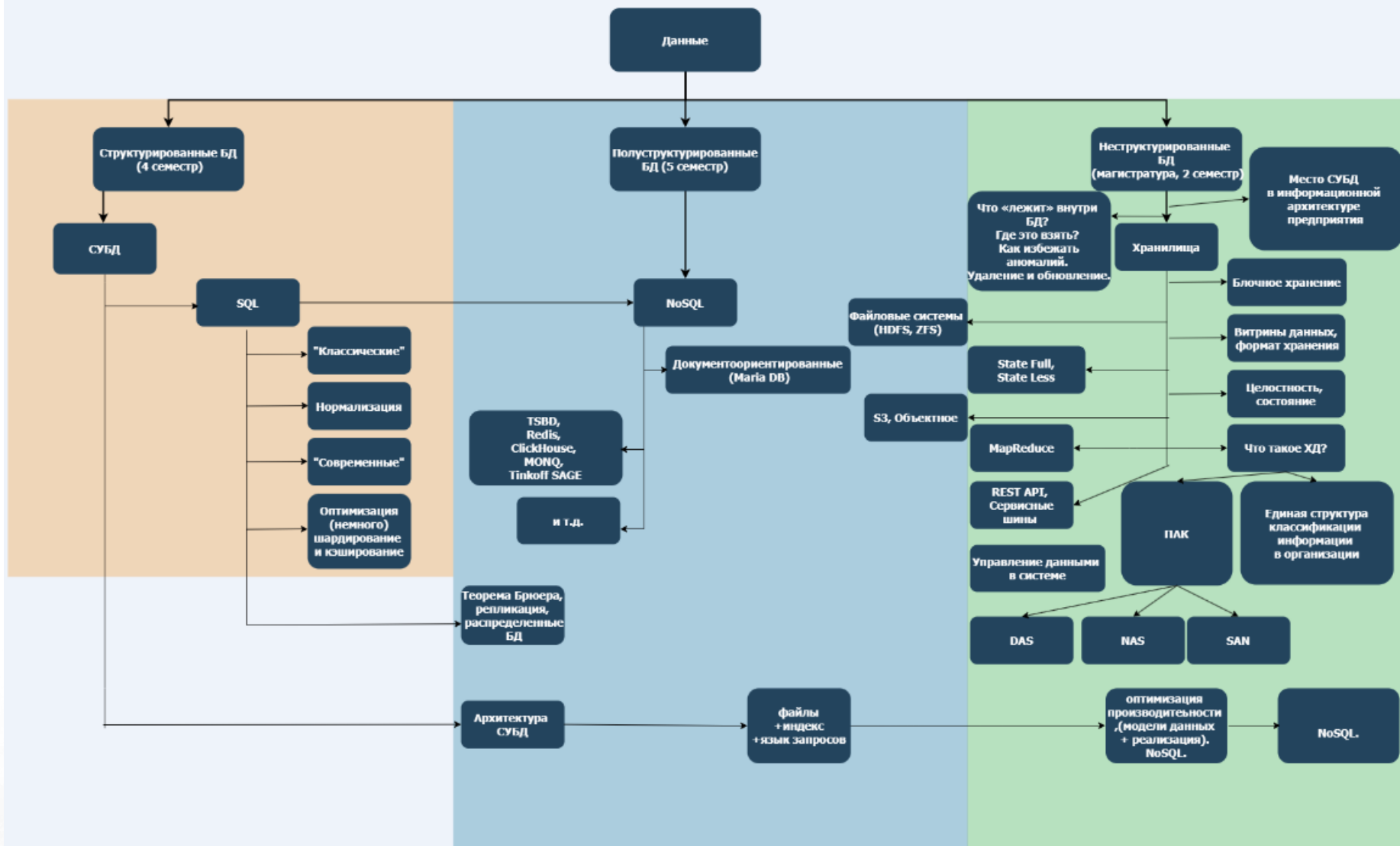
За тест студент может набрать 20 баллов, за которые можно получить:

- От 10 до 14 баллов — оценка «3»;
- От 15 до 19 баллов — оценка «4»;
- 20 баллов — оценка «5».

К баллам за тест прибавляются доп. баллы, которые студент получает в течение семестра по данным критериям:

- Набрано 55-64 балла за семестр — 1 доп.;
- Набрано 65-74 балла за семестр — 2 доп.;
- Набрано 75-84 балла за семестр — 3 доп.;
- Набрано 85-94 балла за семестр — 4 доп.;
- Набрано 95-100 балла за семестр — 5 доп.

В случае, если студент сдал тест на «2», то отправляется на пересдачу без учета доп. баллов.



Заметки:
Бакалавриат:

Проектирование БД (2 курс): Классические структурированные БД,
СУБД = приложение / БП (фактические информационные сущности).

Бакалавриат:

Разработка БД (3 курс) Упор на стройку СУБД в конечное приложение:

- 1) Место СУБД в архитектуре современных **приложений**.
- 2) Нестандартные СУБД (NoSQL, S3 Хранилища...).
- 3) Распределенные БД (шардирование, кэширование, проблема удаленного узла).

Управление данными в приложении:

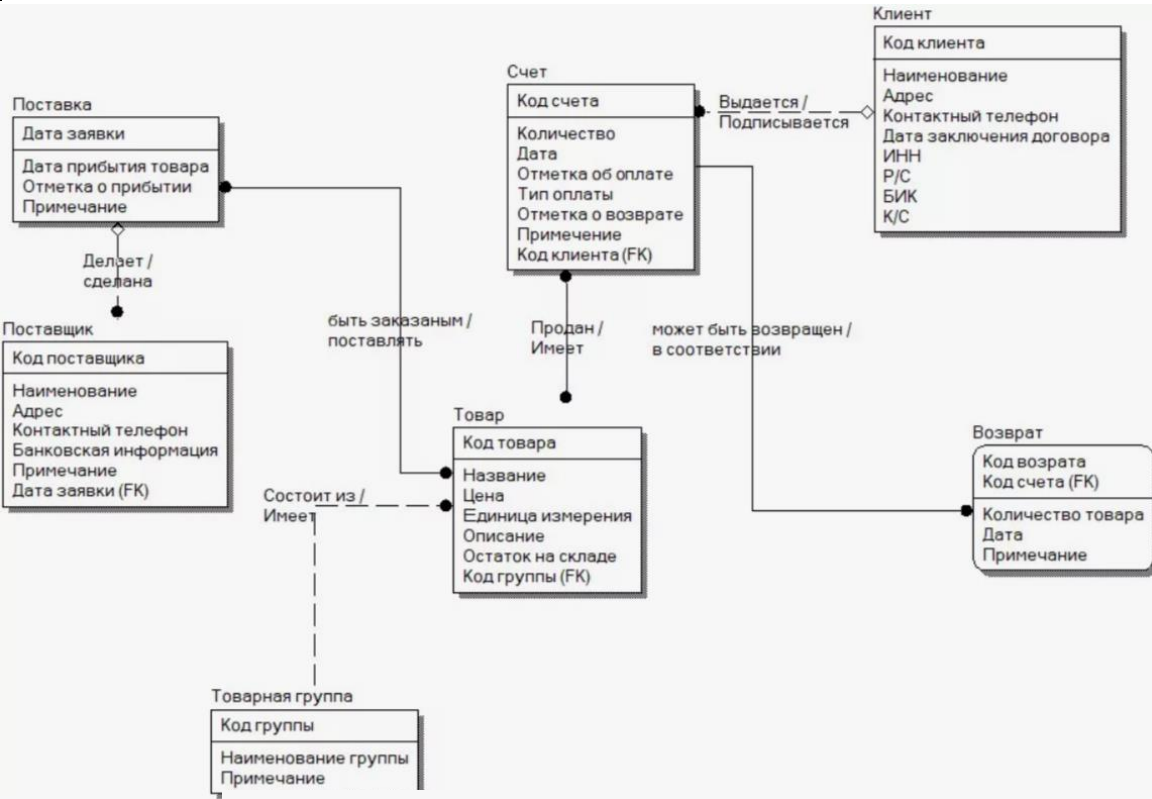
- 1) Физический. Блочное ХД.
- 2) Файловая система.
- 3) Объект.
- 4) СУБД (или набор СУБД).
- 5) Приложение.

Магистратура:

Проектирование
и разработка баз и хранилищ данных (магистратура 1 курс):

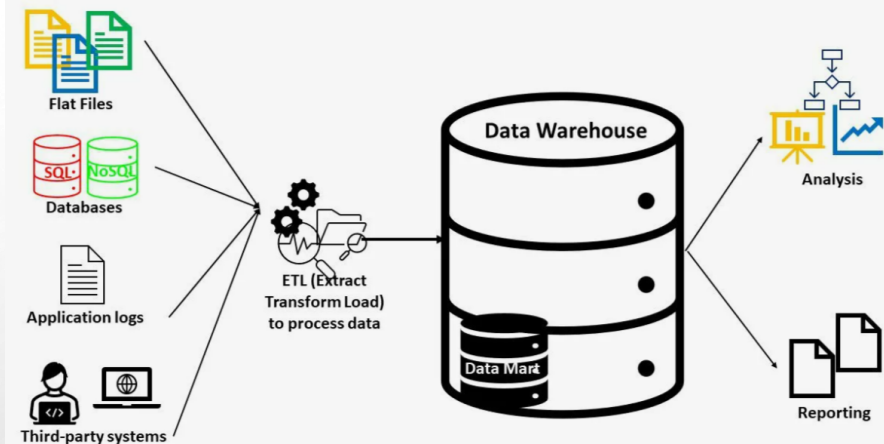
- 1) Место СУБД в информационной архитектуре **предприятия**.
- 2) **Онтологическая модель. Что «лежит» внутри БД? Где это взять?**
Как избежать аномалий. Удаление и обновление.
- 3) **Единая структура классификации.**

MapReduce. NoSQL, State Less, State Full, состояние, целостность данных.



2 курс. Проектирование баз данных

3 курс. Разработка баз данных



Магистратура Проектирование и разработка баз и хранилищ данных

Тема 8_ Типы данных JSON/JSONB, BYTEA. Управление правами доступа. Масштабирование баз данных

8.1_Тип данных JSON/JSONB	12
8.2_Тип данных BYTEA	22
8.3_Управление правами доступа	28
8.4_Масштабирование баз данных	36
Партиционирование	37
Репликация	49

Типы данных JSON и JSONB

PostgreSQL поддерживает формат данных JSON, позволяя использовать его как полноценный тип данных в SQL-запросах и предлагает два типа для хранения: **JSON** и **JSONB**.

```
{  
  "user": {  
    "id": 42,  
    "name": "Иван"  
  },  
  "roles": ["admin", "editor"],  
  "active": true  
}
```


Типы данных JSON и JSONB

	Тип данных JSON	Тип данных JSONB
Хранение	Текстовая строка, содержащая копию исходной строки, включая пробелы и порядок ключей.	Оптимизированное двоичное представление.
Дубликаты ключей	Сохраняются при вводе, но функции используют последнее значение.	Остается только последнее значение.
Скорость записи	Быстрая, выполняется простая проверка на валидность JSON.	Медленная (парсинг, удаление управляющих символов, дубликатов, сортировка ключей и преобразование в двоичный формат)
Скорость запросов	Медленная	Быстрая (благодаря индексам и двоичному формату)
Индексирование	Запросы не индексируются.	Поддерживает индексы GIN и GIST

Примитивные типы JSON и соответствующие им типы в PostgresPro

Примитивный тип JSON	Тип Postgres Pro	Примечания
string	text	\u0000 не допускается как спецпоследовательность Unicode, представляющая символ, который отсутствует в кодировке базы
number	numeric	Значения NaN и infinity не допускаются
boolean	boolean	Допускаются только варианты true и false (в нижнем регистре)
null	(нет)	NULL в SQL имеет другой смысл

JSONB отклоняет числа, которые находятся за пределами диапазона numeric типа данных PostgreSQL.

Примитивные типы JSON и соответствующие им типы в PostgresPro

```
SELECT content FROM book_orders;
```

```
CREATE TABLE book_orders (  
    id serial NOT NULL PRIMARY KEY,  
    content jsonb NOT NULL  
);
```

	content jsonb 
1	<code>{"items": {"qty": 3, "book": "Hyperion"}, "customer": "Jacob Johnson"}</code>
2	<code>{"items": {"qty": 2, "book": "War and Peace"}, "customer": "Adam Brown"}</code>
3	<code>{"items": {"qty": 4, "book": "1984"}, "customer": "Andrew Nelson"}</code>

```
INSERT INTO book_orders (content)  
VALUES('{"customer": "Jacob Johnson", "items": {"book": "Hyperion", "qty": 3}}'),  
      ('{"customer": "Adam Brown", "items": {"book": "War and Peace", "qty": 2}}'),  
      ('{"customer": "Andrew Nelson", "items": {"book": "1984", "qty": 4}}');
```

Операции с JSON. Извлечение значений

Операторы -> и ->> позволяют получить значение из объекта по ключу:

-> — возвращает значение в виде JSON;

->> — возвращает значение как текст.

Получить имена всех покупателей в виде объектов JSON и в текстовом виде:

SELECT

content -> 'customer' AS customer

FROM book_orders;

	customer jsonb 
1	"Jacob Johnson"
2	"Adam Brown"
3	"Andrew Nelson"

SELECT

content ->> 'customer' AS customer

FROM book_orders;

	customer text 
1	Jacob Johnson
2	Adam Brown
3	Andrew Nelson

Операции с JSON. Извлечение вложенных значений

Чтобы получить значение из вложенного объекта, можно использовать цепочку операторов.

Пример. Вернуть все заказанные книги:

```
SELECT content -> 'items' ->> 'book' AS book  
FROM book_orders;
```

	book text 
1	Hyperion
2	War and Peace
3	1984

Также доступны операторы #> и #>>, которые принимают путь в виде массива:

#> — возвращает JSON; #>> — возвращает текст.

Пример. Вернуть все заказанные книги:

```
SELECT content #> '{items, book}' AS books  
FROM book_orders;
```

	books jsonb 
1	"Hyperion"
2	"War and Peace"
3	"1984"

Фильтрация и поиск по ключу

Оператор @> проверяет, содержит ли JSON-объект указанный фрагмент.

Работает только с типом jsonb.

```
SELECT * FROM book_orders
```

```
WHERE content @> '{"customer": "Adam Brown"}';
```

id [PK] integer	content jsonb
2	{"items": {"qty": 2, "book": "War and Peace"}, "customer": "Adam Brown"}

Для поиска по ключу используются операторы:

? — проверяет наличие одного ключа;

?| — проверяет наличие хотя бы одного из списка ключей.

Проверяем наличие ключа 'customer' в каждой строке таблицы:

```
SELECT content ? 'customer' FROM book_orders;
```

Проверяем наличие ключа 'customer' в каждой строке таблицы:

```
SELECT id FROM book_orders WHERE content ?| array['email', 'phone'];
```

Применение агрегатных функций к JSON

Пример. Вернуть минимальное, максимальное, среднее и общее количество книг.

SELECT

ROUND(AVG (CAST (content -> 'items' ->> 'qty' AS INTEGER)), 2) AS AVG,

MIN (CAST (content -> 'items' ->> 'qty' AS INTEGER)),

MAX (CAST (content -> 'items' ->> 'qty' AS INTEGER)),

SUM (CAST (content -> 'items' ->> 'qty' AS INTEGER))

FROM book_orders;

	avg numeric 	min integer 	max integer 	sum bigint 
1	3.00	2	4	9

Функции для работы с JSON/JSONB

Перебор ключей и значений

Функция `json_each` (`jsonb_each`) разбивает

JSON-объект на пары «ключ – значение»:

```
SELECT jsonb_each(content) FROM book_orders;
```

	jsonb_each record
1	items,{"qty": 3, "book": "Hyperion"}
2	customer,"Jacob Johnson"
3	items,{"qty": 2, "book": "War and Peace"}
4	customer,"Adam Brown"
5	items,{"qty": 4, "book": "1984"}
6	customer,"Andrew Nelson"

Получение списка ключей

`json_object_keys` (`jsonb_object_keys`) возвращает только список

ключей верхнего уровня в JSON-объекте:

```
SELECT jsonb_object_keys (content->'items') FROM book_orders;
```

	jsonb_object_keys text
1	qty
2	book
3	qty
4	book
5	qty
6	book

Функции для работы с JSON/JSONB

Определение типа значения

`json_typeof` (`jsonb_typeof`) определяет тип значения, находящегося по указанному ключу. Результатом будет строка: `object`, `array`, `string`, `number`, `boolean` или `null`:

```
SELECT jsonb_typeof (content->'items'->'qty') FROM book_orders;
```

Извлечение вложенных значений по пути

`json_extract_path` (`jsonb_extract_path`) извлекает вложенное значение по пути из ключей:

```
SELECT jsonb_extract_path (content, 'items','qty') FROM book_orders;
```

Тип данных BYTEA

BYTEA (Binary String) - это основной тип данных для хранения двоичных данных в PostgreSQL. Он позволяет хранить двоичные строки переменной длины. Двоичная строка представляет собой последовательность октетов (байтов).

Сравнение:

Двоичные строки	Строки символов
Может хранить любые байты, включая нулевые (0x00).	Могут хранить только те последовательности байтов, которые являются допустимыми символами в кодировке базы данных
Операции с двоичными строками обрабатывают байты	Обработка символьных строк зависит от настроек языковых стандартов
Файлы (изображения, PDF), сериализованные объекты, хеши.	Тексты на естественных языках

Тип данных BYTEA

BYTEA поддерживает два формата ввода и вывода:

- **шестнадцатеричный формат** - **hex (по умолчанию)**
- **формат спецпоследовательностей** - **escape**

Формат hex кодирует двоичные данные двумя шестнадцатеричными цифрами на байт.

Формат: `'\x<binary string>'`

Пример: `SELECT '\xDEADBEEF';` или `SELECT '\xDE AD BE EF';`

Формат escape (формат спецпоследовательностей) традиционно использовался в Postgres Pro для значений типа `bytea`.

В нём двоичная строка представляется в виде последовательности ASCII-символов, а байты, непредставимые в виде ASCII-символов, передаются в виде спецпоследовательностей - конвертируются в код вида `\xxx`, где `xxx` — это трёхзначное восьмеричное число.

Функции преобразования текстовых/двоичных строк

Функция	Назначение	Пример использования
<code>convert_from (<двоичная_строка> bytea, <исходная_кодировка>) → text</code>	Интерпретирует двоичные данные как текст в указанной кодировке.	<code>convert_from('\x746578745f696e5f75746638', 'UTF8');</code> Результат: "text_in_utf8"
<code>convert_to (<текстовая_строка> text, <результатирующая_кодировка>) → bytea</code>	Преобразует текст в двоичные данные указанной кодировки.	<code>convert_to('some_text', 'UTF8');</code> Результат: \x736f6d655f74657874.
<code>encode (<двоичная_строка> bytea, <формат> text) → text</code>	преобразует двоичные данные в текстовое представление.	<code>encode('\x41445047', 'base64');</code> Результат: QURQRw==
<code>decode (<текстовая_строка> text, <формат> text) → bytea</code>	Преобразует текстовое представление в двоичные данные	<code>decode('QURQRw==', 'base64');</code> Результат: \x41445047.

Функции для работы с двоичными строками

Функция	Назначение	Пример использования
<code>string string -> bytea</code>	Конкатенация строк	<code>'\x123456'::bytea '\x789a00bcde'::bytea;</code> Результат: <code>\x123456789a00bcde</code> .
<code>octet_length(string) -> int</code>	Число байт в двоичной строке	<code>octet_length('jo\000se'::bytea)</code> Результат: 5
<code>position(substring in string) -> int</code>	возвращает начальный индекс первой найденной подстроки в байтах или ноль, если подстрока не найдена	<code>position('\x5678'::bytea in '\x1234567890'::bytea);</code> Результат: 3.
<code>substring(string [from int] [for int]) -> bytea</code>	извлекает подстроку, которая начинается с указанного байта и имеет указанную длину в байтах.	<code>substring('\x1234567890'::bytea from 3 for 2);</code> Результат: <code>\x5678</code> .

Работа с типом данных BYTEA

Пример. Создание таблицы, в которой хранятся название файла и графическое изображение в двоичном формате

```
CREATE TABLE documents (  
    id SERIAL PRIMARY KEY,  
    filename text NOT NULL,  
    file_data BYTEA,  
    file_size INTEGER,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Добавление данных:

```
INSERT INTO documents (filename, file_data, file_size)  
VALUES ('image1.jpg', '\x89504e470d0a1a0a00000000d49484452', 16);
```

Выборка:

```
SELECT id, filename, encode(file_data, 'hex') as file_content FROM documents;
```

Подготовленные операторы - PREPARE

PREPARE создаёт подготовленный оператор.

Синтаксис:

PREPARE имя [(тип_данных [, ...])] AS оператор

```
PREPARE documents_plan(text, bytea, integer) AS  
            INSERT INTO documents (filename, file_data, file_size) VALUES ($1, $2, $3);
```

```
-----  
EXECUTE documents_plan('image2.jpg', '\x89504e470d0a1a0a0000000d49484452', 16);
```

Освободить план:

```
DEALLOCATE documents_plan;
```

Управление ролями пользователей

Роль представляет собой пользователя базы данных или группу пользователей, которые могут владеть объектами и иметь определённые права.

CREATE ROLE добавляет новую роль в кластер баз данных PostgreSQL:

CREATE ROLE имя [[WITH] параметр [...]]

Параметры: **CREATEDB** - определяет, сможет ли роль создавать базы данных.

CREATEROLE определяет, сможет ли роль создавать, изменять и удалять другие роли, добавлять для них комментарии и изменять метку безопасности в любых ролях.

LOGIN определяет, может ли эта роль стать начальным авторизованным именем при подключении клиента. Если NOLOGIN — роль не является пользовательской и не сможет подключиться к серверу

PASSWORD 'пароль' - задаёт пароль роли.

VALID UNTIL 'дата_время' устанавливает дату и время, после которого пароль роли перестает действовать.

Управление ролями

1. Создание роли user1 с паролем password1

```
CREATE ROLE user1 WITH LOGIN PASSWORD 'password1';
```

2. Представление pg_roles предоставляет доступ к информации о ролях в базах данных

```
SELECT rolname FROM pg_roles;
```

3. Изменение атрибутов роли после создания:

- Изменение пароля роли user1:

```
ALTER ROLE user1 WITH PASSWORD 'password2';
```

- Добавление привилегии CREATEDB и CREATEROLE роли user2:

```
ALTER ROLE user2 CREATEROLE CREATEDB;
```

3. Группировка пользователей для управления привилегиями

```
CREATE ROLE group1 CREATEROLE CREATEDB;
```

Управление правами доступа

DCL (Data Control Language) — это часть языка SQL, которая используется для управления правами доступа в базах данных с помощью команд **GRANT** (предоставить) и **REVOKE** (отозвать).



Предоставить права пользователю:

```
GRANT { { SELECT | INSERT | UPDATE | REFERENCES } ( имя_столбца [, ...] )
```

```
[, ...] | ALL [ PRIVILEGES ] ( имя_столбца [, ...] ) }
```

```
ON [ TABLE ] имя_таблицы [, ...]
```

```
TO указание_роли [, ...] [ WITH GRANT OPTION ]
```

```
[ GRANTED BY указание_роли ]
```

Управление правами доступа

GRANT для объектов баз данных - даёт одной или нескольким ролям определённые права для доступа к объекту базы данных.

Разрешает всем добавлять записи в таблицу films:

```
GRANT INSERT ON films TO PUBLIC;
```

Предоставление прав на создание схем для роли db_creator

```
GRANT CREATE ON DATABASE mydb TO db_creator;
```

Предоставление прав на чтение для роли read_only_group

```
GRANT SELECT ON ALL TABLES IN SCHEMA public TO read_only_group;
```

Предоставление полных прав для роли write_group

```
GRANT SELECT, INSERT, UPDATE, DELETE ON table_name TO write_group;
```

Управление правами доступа

GRANT для ролей - эта разновидность команды GRANT включает роль в члены одной или нескольких других ролей, а также изменяет параметры членства в роли.

1. Выдать пользователю manuel все права на kinds

```
GRANT ALL PRIVILEGES ON kinds TO manuel;
```

2. Включение в роль admin пользователя employees:

```
GRANT admin TO employees;
```

3. Создаем роль без наследования

```
CREATE ROLE admin_role NOINHERIT;
```

```
GRANT admin_role TO app_user;
```

```
SET ROLE admin_role;
```

```
RESET ROLE;
```

Управление правами доступа

Пример управления правами доступа

1. Создаем групповые роли

-CREATE ROLE read_only;

-CREATE ROLE admin;

2. Создаем пользователей

DO \$\$

BEGIN

IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'reporter') THEN

CREATE USER reporter WITH PASSWORD 'reporter_pass';

END IF;

IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = 'manager') THEN

CREATE USER manager WITH PASSWORD 'manager_pass';

END IF;

END

\$\$;

3. Назначаем пользователем групповых ролей

-GRANT read_only TO reporter;

-GRANT admin TO manager;

Управление правами доступа

4. Назначаем привилегии для групповых ролей

На подключение к БД:

- GRANT CONNECT ON DATABASE shop TO read_only, admin;**
- GRANT USAGE ON SCHEMA public TO read_only, admin;**
- GRANT CREATE ON SCHEMA public TO admin;**

На существующие таблицы

- GRANT SELECT ON ALL TABLES IN SCHEMA public TO read_only;**
- GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin;**

На последовательности:

- GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO admin;**
- GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO read_only;**

На будущие объекты

- ALTER DEFAULT PRIVILEGES FOR USER postgres IN SCHEMA public**
GRANT SELECT ON TABLES TO read_only;
- ALTER DEFAULT PRIVILEGES FOR USER postgres IN SCHEMA public**
GRANT SELECT ON SEQUENCES TO read_only;

Управление правами доступа

REVOKE — отозвать права доступа

REVOKE [GRANT OPTION FOR]

{ { SELECT | INSERT | UPDATE | REFERENCES }

(имя_столбца [, ...])

[, ...] | ALL [PRIVILEGES] (имя_столбца [, ...]) }

ON [TABLE] имя_таблицы [, ...]

FROM указание_роли [, ...]

[GRANTED BY указание_роли]

[CASCADE | RESTRICT]

Удалить у группы public права добавлять данные в таблицу films:

REVOKE INSERT ON films FROM PUBLIC;

Назначить владельцем таблицы базы данных table1 роль user2.

ALTER TABLE table1 OWNER TO user2;

Команда REASSIGN OWNED передает объекты базы данных новому владельцу, но не влияет на предоставленные привилегии для объектов, не принадлежащих роли.

REASSIGN OWNED BY user1 TO user2;

Удалить привилегии

DROP OWNED BY user1;

Удалить роль

DROP ROLE user1;

Масштабирование баз данных

Вертикальное масштабирование – это процесс увеличения мощности одного сервера путем добавления к нему более производительных комплектующих (CPU, память).

Горизонтальное масштабирование — это метод повышения производительности за счет добавления новых узлов для распределения нагрузки и данных между ними.

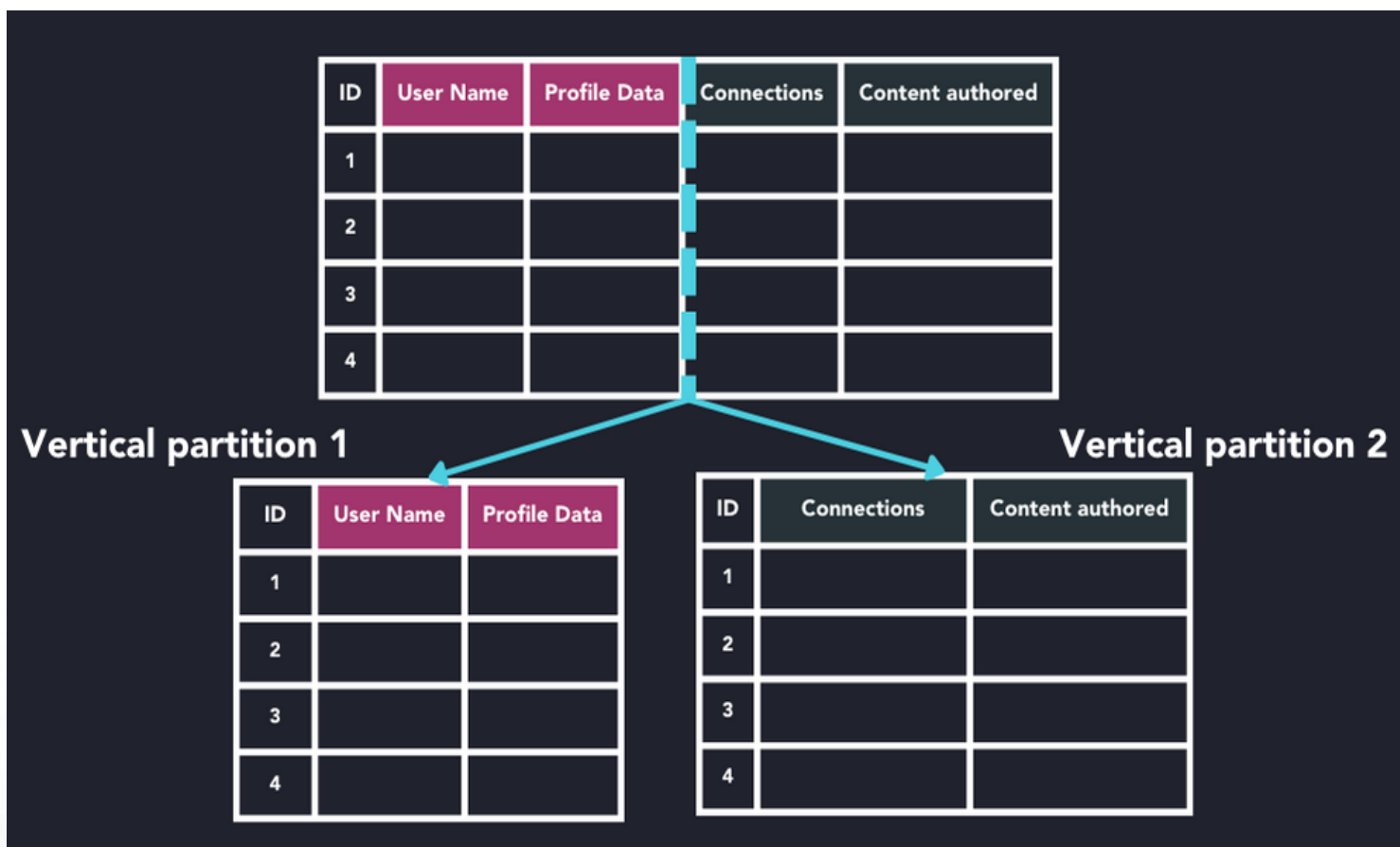
Стратегии горизонтального масштабирования: ***партиционирование (секционирование), репликацию, шардирование.***

Партицирование базы данных

Партицирование(секционирование) базы данных — это метод разбиения большой таблицы в рамках единой базы на более мелкие, логически связанные части - партиции.

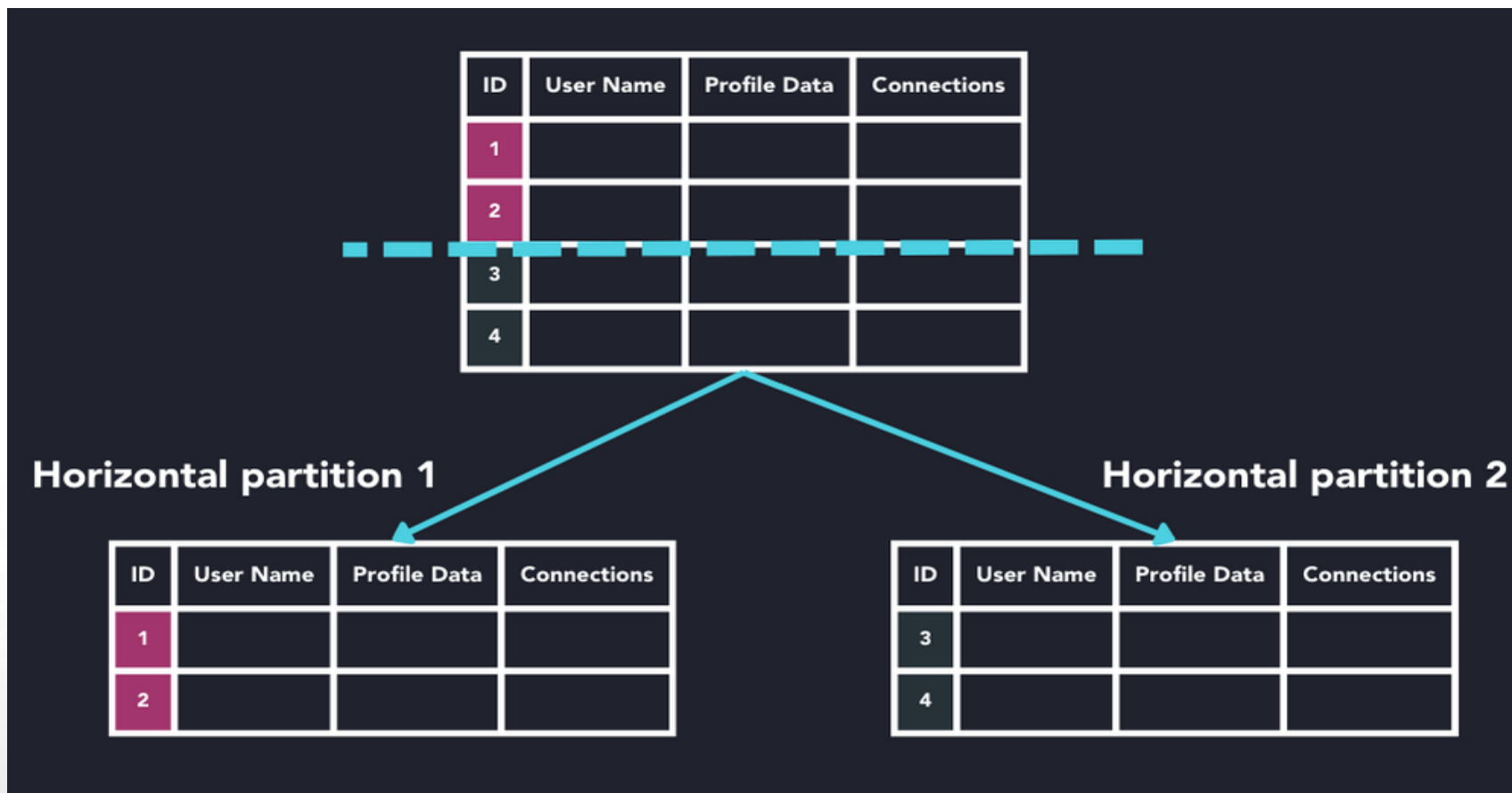
Каждая партиция хранится отдельно, но логически объединяется в одну таблицу, а база автоматически направляет запросы в каждую партицию.

Вертикальное партицирование —таблица разделяется на партиции по принципу выбранных колонок.



Горизонтальное партицирование базы данных

Горизонтальное партицирование — метод, заключающийся в делении таблицы по строкам.



Секционирование в PostgresPro

PostgreSQL поддерживает **декларативное секционирование** таблиц

Метод	Описание
Секционирование по диапазонам	Таблица секционируется по диапазонам, определенным по ключевому столбцу или набору столбцов. Диапазоны не должны пересекаться друг с другом. Например, можно секционировать данные по диапазонам дат или по диапазонам идентификаторов определенных бизнес-объектов. Границы каждого диапазона включают нижнее значение и исключают верхнее.
Секционирование по списку	Таблица разделяется с помощью списка, явно указывающего, какие значения ключа должны относиться к каждой секции
Секционирование по хешу	Таблица секционируется по определенным модулям и остаткам, которые указываются для каждой секции. Каждая секция содержит строки, для которых хеш-значение ключа разбиения, разделенное на модуль, равняется заданному остатку

Секционирование в PostgresPro

Секционированная таблица — это виртуальная таблица, в которой нет строк.

Секции — это обычные таблицы, связанные с секционированной:

- Каждая секция хранит подмножество строк, определяемое значениями ключа секции.
- PostgreSQL вставляет строки в соответствующую секцию на основе ключа секционирования.
- Если в строке обновляется значение ключа секции и он больше не соответствует значениям секции, строка перемещается в другую секцию.
- Невозможно преобразовать обычную таблицу в секционированную или наоборот. Однако можно добавить существующую таблицу в качестве секции к секционированной таблице или удалить секцию и превратить её в отдельную таблицу.

Секционирование в PostgresPro

Синтаксис создания секционированной таблицы:

```
CREATE <имя_таблицы>  
PARTITION BY { RANGE | LIST | HASH }  
( { <имя_столбца> | ( <выражение> ) } )
```

Синтаксис создания секций:

```
CREATE TABLE <имя_секции> PARTITION OF <имя_таблицы>  
  { FOR VALUES FROM (<минимальное_значение>  
    TO (<максимальное_значение>  
    | FOR VALUES IN ([<значение>])  
    | FOR VALUES WITH (MODULUS <значение_модуля>, REMAINDER  
    <значение_остатка>) }
```

Секционирование по диапазонам

Пример. Создаем три секции и распределяем книги в соответствии со значением поля `delivery_date`

```
CREATE TABLE books (  
    code      char(5) NOT NULL,  
    title     varchar(40) NOT NULL,  
    delivery_date date NOT NULL,  
    genre     varchar(10)  
) PARTITION BY RANGE (delivery_date);
```

```
CREATE TABLE books_2022_02 PARTITION OF books  
    FOR VALUES FROM ('2022-02-01') TO ('2022-03-01');  
CREATE TABLE books_2022_03 PARTITION OF books  
    FOR VALUES FROM ('2022-03-01') TO ('2022-04-01');  
CREATE TABLE books_2022_04 PARTITION OF books  
    FOR VALUES FROM ('2022-04-01') TO ('2022-05-01');  
CREATE TABLE books_default PARTITION OF books DEFAULT;
```

Секционирование по диапазонам

Добавляем книгу, она попадет в секцию books_2022_04:

```
INSERT INTO books (code, title, delivery_date, genre) VALUES  
('DC-34','Hyperion', date'2022-04-28', 'sci-fi');
```

Использование в запросах:

```
SELECT title FROM books WHERE EXTRACT(YEAR FROM delivery_date) = 2023;
```

Использование специальных значений MINVALUE и MAXVALUE, обозначающих минимальное и максимальное значения, содержащиеся в столбце, при указании диапазонов:

```
CREATE TABLE table1_part1 PARTITION OF table1 FOR VALUES FROM (MINVALUE) TO (100);  
CREATE TABLE table1_part2 PARTITION OF table1 FOR VALUES FROM (100) TO (1000);  
CREATE TABLE table1_part3 PARTITION OF table1 FOR VALUES FROM (1000) TO (MAXVALUE);
```

Секционирование по спискам

Пример. Секционирование по жанрам

```
CREATE TABLE books (  
    code    char(5) NOT NULL,  title    varchar(40) NOT NULL,  
    delivery_date date NOT NULL, genre    varchar(10) NOT NULL  
    ) PARTITION BY LIST (genre);
```

```
CREATE TABLE books_novel PARTITION OF books FOR VALUES IN ('novel');
```

```
CREATE TABLE books_scifi PARTITION OF books FOR VALUES IN ('sci-fi');
```

```
CREATE TABLE books_detective PARTITION OF books FOR VALUES IN ('detective', 'mystery');
```

```
CREATE TABLE books_default PARTITION OF books DEFAULT;
```

Секционирование по хешу

Пример. Секционирование по хешу. Используется, если нужно равномерно распределить строки по секциям, а таблица не имеет подходящего ключа секционирования:

```
CREATE TABLE books (  
    code    char(5) NOT NULL,  title    varchar(40) NOT NULL,  
    delivery_date date NOT NULL, genre    varchar(10) NOT NULL  
    ) PARTITION BY HASH (code);
```

```
CREATE TABLE books_part1 PARTITION OF books FOR VALUES WITH (MODULUS 5, REMAINDER 0);  
CREATE TABLE books_part2 PARTITION OF books FOR VALUES WITH (MODULUS 5, REMAINDER 1);  
CREATE TABLE books_part3 PARTITION OF books FOR VALUES WITH (MODULUS 5, REMAINDER 2);  
CREATE TABLE books_part4 PARTITION OF books FOR VALUES WITH (MODULUS 5, REMAINDER 3);  
CREATE TABLE books_part5 PARTITION OF books FOR VALUES WITH (MODULUS 5, REMAINDER 4);
```

Вложенное секционирование

Таблицы-секции могут быть также разбиты на секции.

Пример. Секцию `books_scifi` разделим еще на три по диапазону дат:

```
CREATE TABLE books_scifi PARTITION OF books FOR VALUES IN ('sci-fi')
                                PARTITION BY RANGE (delivery_date);
CREATE TABLE books_scifi_2022_01 PARTITION OF books_scifi
                                FOR VALUES FROM ('2022-01-01') TO ('2022-02-01');
CREATE TABLE books_scifi_2022_02 PARTITION OF books_scifi
                                FOR VALUES FROM ('2022-02-01') TO ('2022-03-01');
CREATE TABLE books_scifi_2022_03 PARTITION OF books_scifi
                                FOR VALUES FROM ('2022-03-01') TO ('2022-04-01');
```

PostgreSQL вставит следующую строку в таблицу **`books_scifi_2022_02`**:

```
INSERT INTO books (code, title, delivery_date, genre) VALUES
                                ('AC-29', 'The time machine', date '2022-02-28', 'sci-fi');
```


Удаление секции

1. Удаления секции (требуется блокировка ACCESS EXCLUSIVE для секционированной таблицы)

DROP TABLE books_novel;

2. Удаление секции из секционированной таблицы, но сохранение к ней доступ как к обычной таблице.

ALTER TABLE books DETACH PARTITION books_novel;

(требует блокировку ACCESS EXCLUSIVE)

ALTER TABLE books DETACH PARTITION books_novel CONCURRENTLY;

(требуется блокировка SHARE UPDATE EXCLUSIVE)

Ограничения декларативного секционирования

1. Столбцы в секциях должны иметь те же имена и типы данных, что и в родительской таблице.
2. Все столбцы, используемые в ключе секционирования, должны быть частью уникального индекса или первичного ключа
3. Триггеры уровня BEFORE ROW для INSERT не могут менять секцию, в которую попадёт новая строка.
4. Ограничения CHECK и NOT NULL, определенные в секционируемой таблице, всегда наследуются всеми её секциями.
5. Нельзя создать индекс, который бы охватывал все секции разом. Каждый индекс создается внутри каждой секции.
6. Интервалы, заданные в FOR VALUES, должны быть непересекающимися. В секционировании по спискам — каждый ключ может принадлежать только одной секции, для секционирования по диапазонам — диапазоны не должны пересекаться.
7. При секционировании по спискам и диапазонам должна существовать секция по умолчанию, чтобы принять строки, которые не подходят ни под одну из существующих секций.

Репликация базы данных

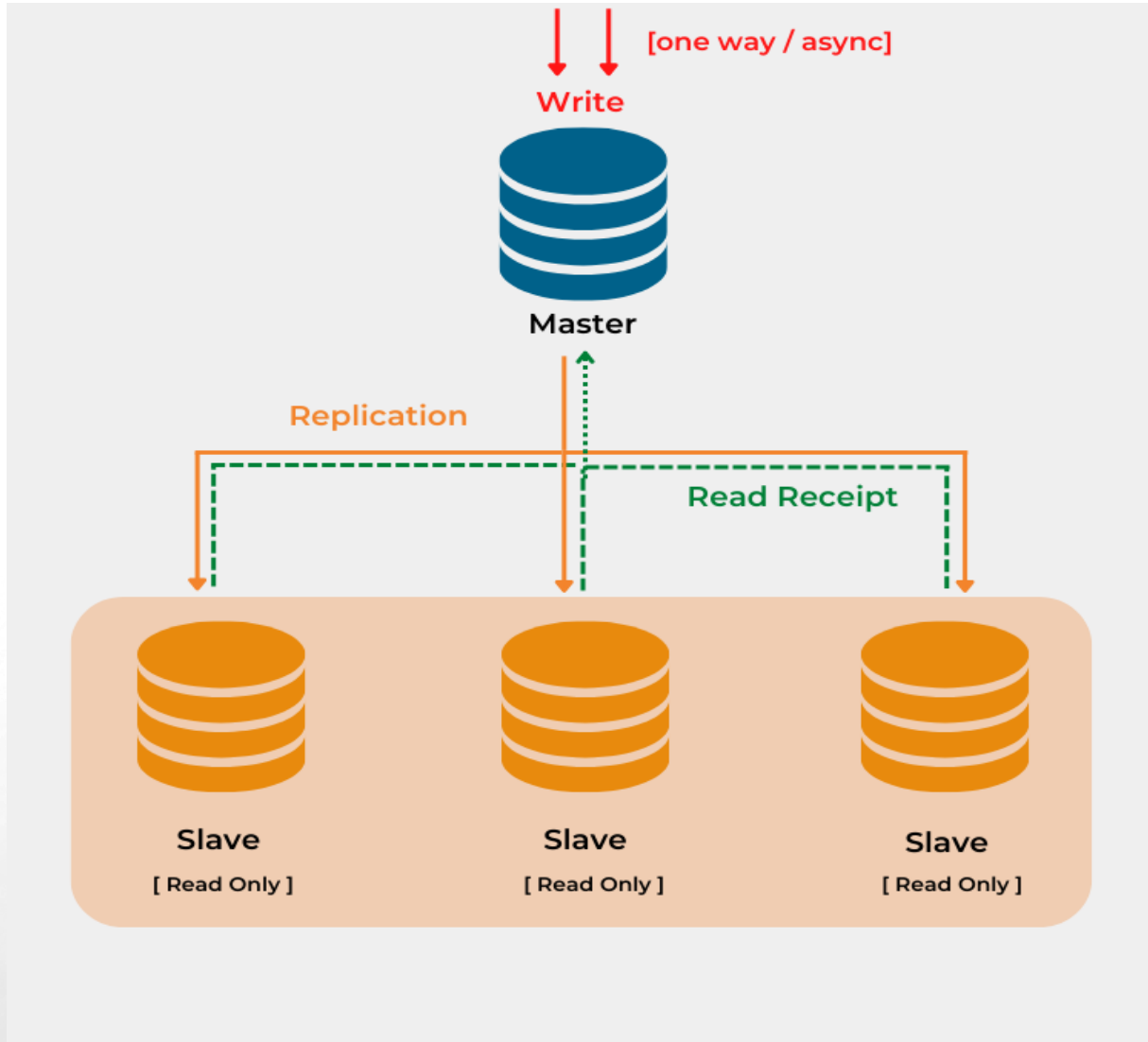
Репликация базы данных — это процесс автоматического копирования и синхронизации данных между несколькими серверами (узлами). При репликации изменения, внесённые в одну копию базы (основной узел), автоматически передаются на другие узлы (реплики), которые могут использоваться для балансировки нагрузки, резервного копирования или аналитики.

Существует две основные схемы репликации: **Master-Slave (ведущий-ведомый)** и **Master-Master (ведущий-ведущий)**:

Master Database (Главная СУБД) — это оригинал. Имеет доступ ко всей информации в соответствии со всеми уровнями запросов

Slave Database (Подчиненная СУБД) — реплика главной СУБД.

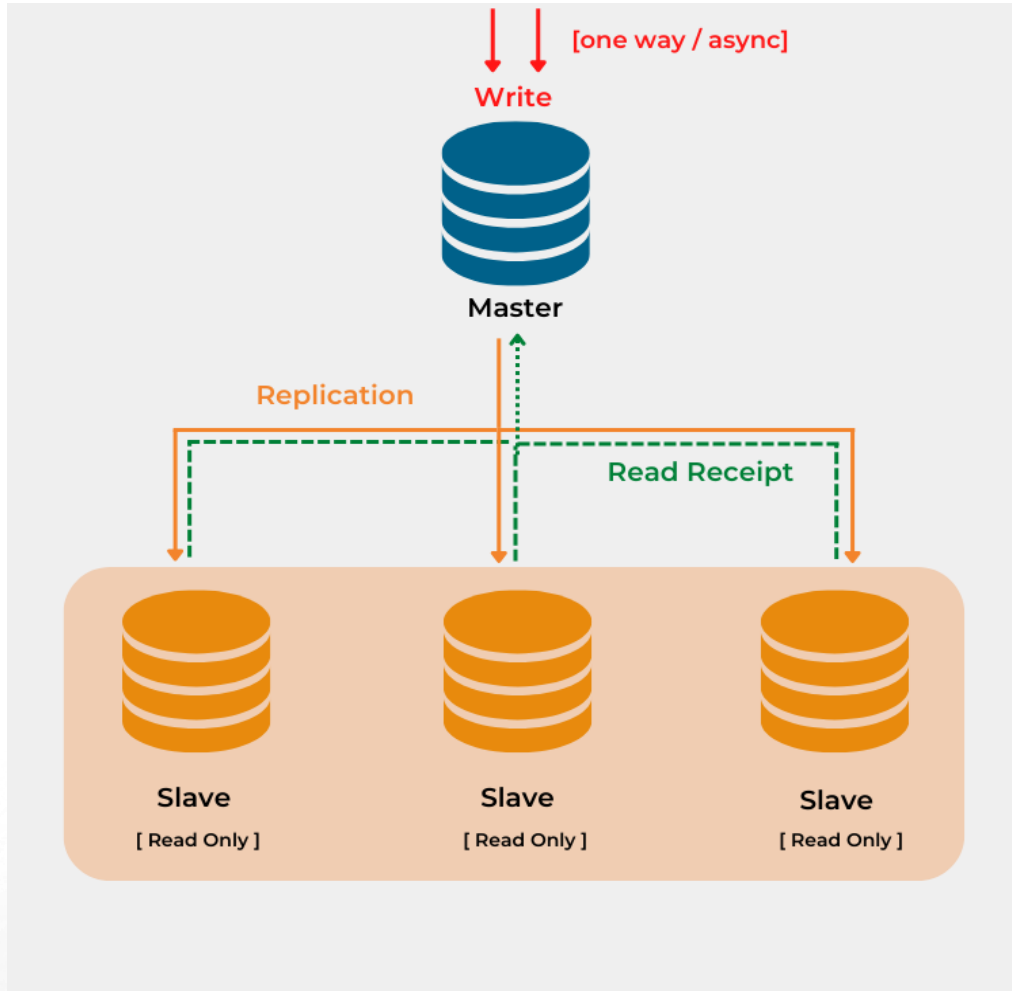
Репликация Master-Slave



Master-Slave репликация — это способ организации базы данных, где есть один главный сервер (Master) и один или несколько подчинённых (Slave):

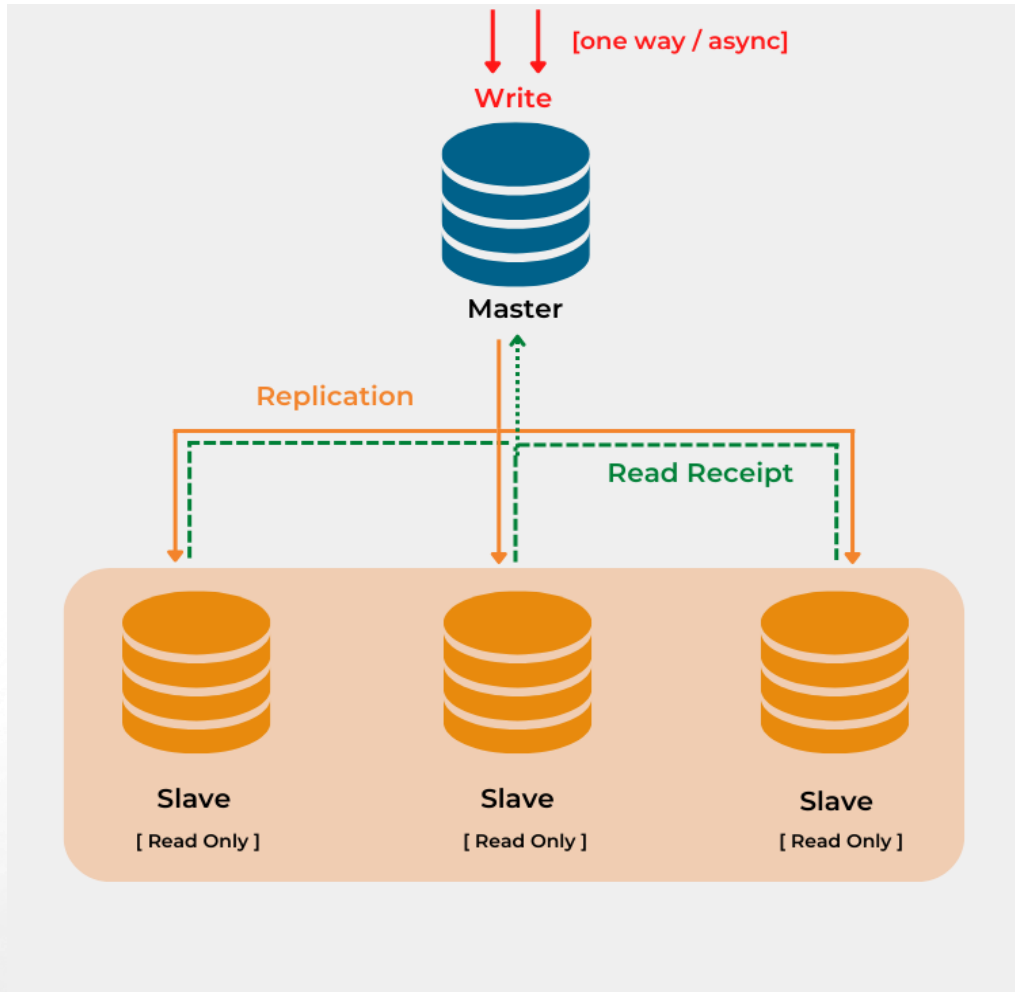
- Master принимает все запросы на запись (добавление, изменение, удаление данных),
- Slaves только читают данные и получают обновления от Master.

Достоинства Master-Slave



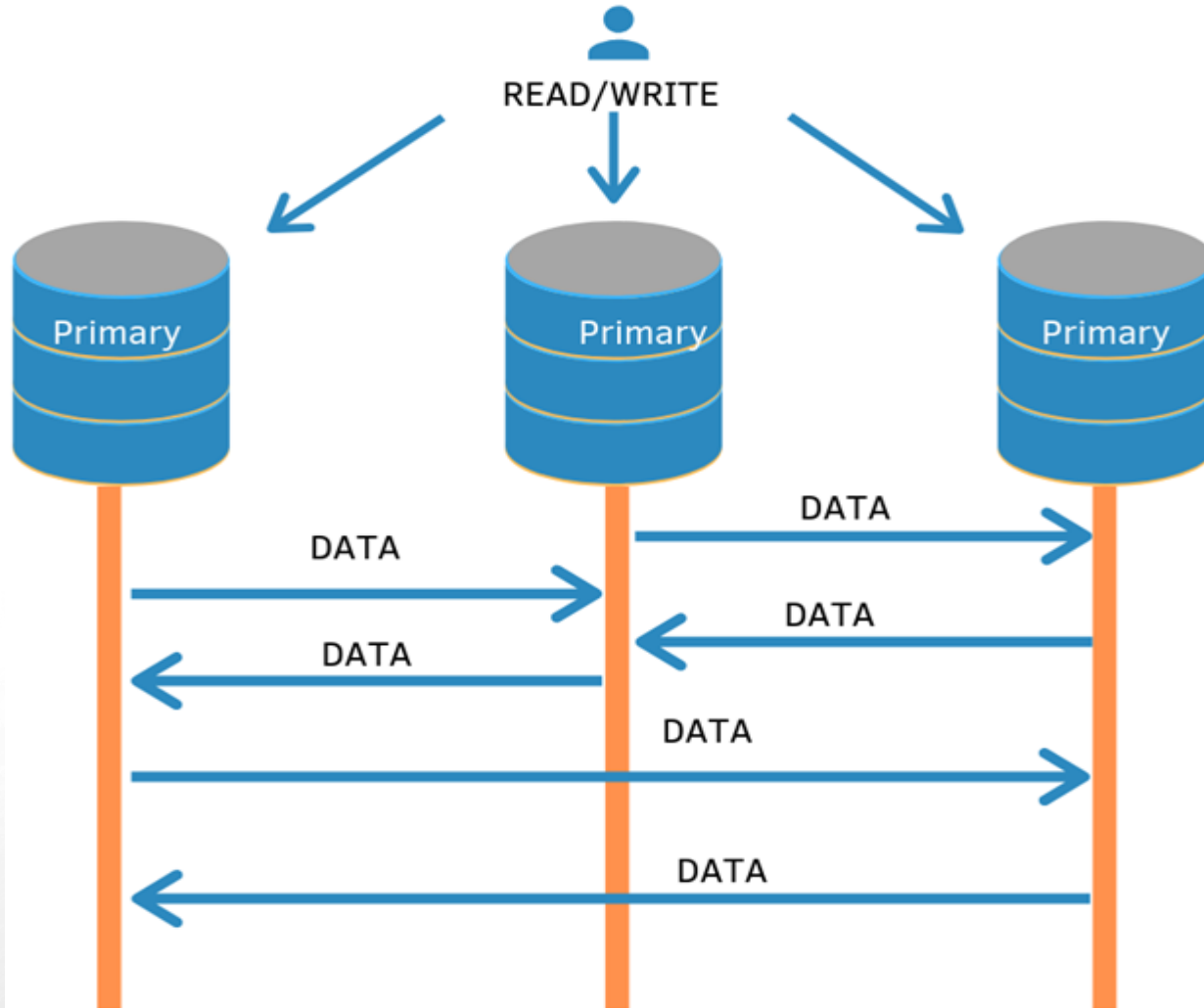
- Нагрузка запросов на чтение распределяется между несколькими репликами, что позволяет обрабатывать больше запросов и повышать общую производительность системы.
- В случае сбоя основного сервера, один из подчинённых (Slave) может быть назначен новым мастером, что минимизирует время простоя.
- Для обработки возросшего числа запросов на чтение можно добавить новые реплики, не нагружая при этом основной сервер

Недостатки Master-Slave



- Потенциальная несогласованность данных: из-за задержки синхронизации реплики могут возвращать устаревшие данные.
- Сложность обеспечения отказоустойчивости: при падении мастера требуется сложная процедура переключения, которая может быть как ручной, так и автоматической.
- Ограничение масштабируемости записи: поскольку все операции записи выполняются только на основном сервере, он становится узким местом при высокой нагрузке на запись.

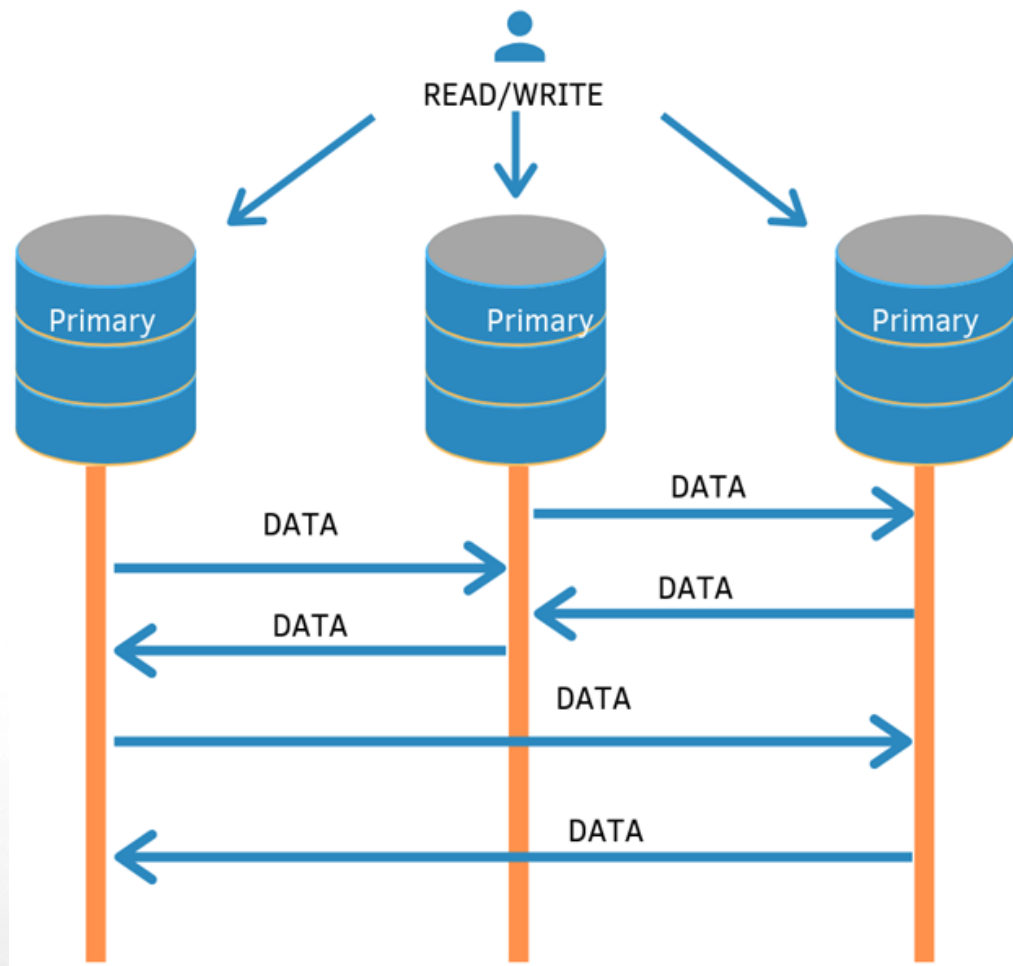
Репликация Master-Master



Master-Master репликация — это способ, при котором у базы данных есть два (или больше) главных сервера, и каждый из них может одновременно принимать и обрабатывать запросы на запись и чтение.

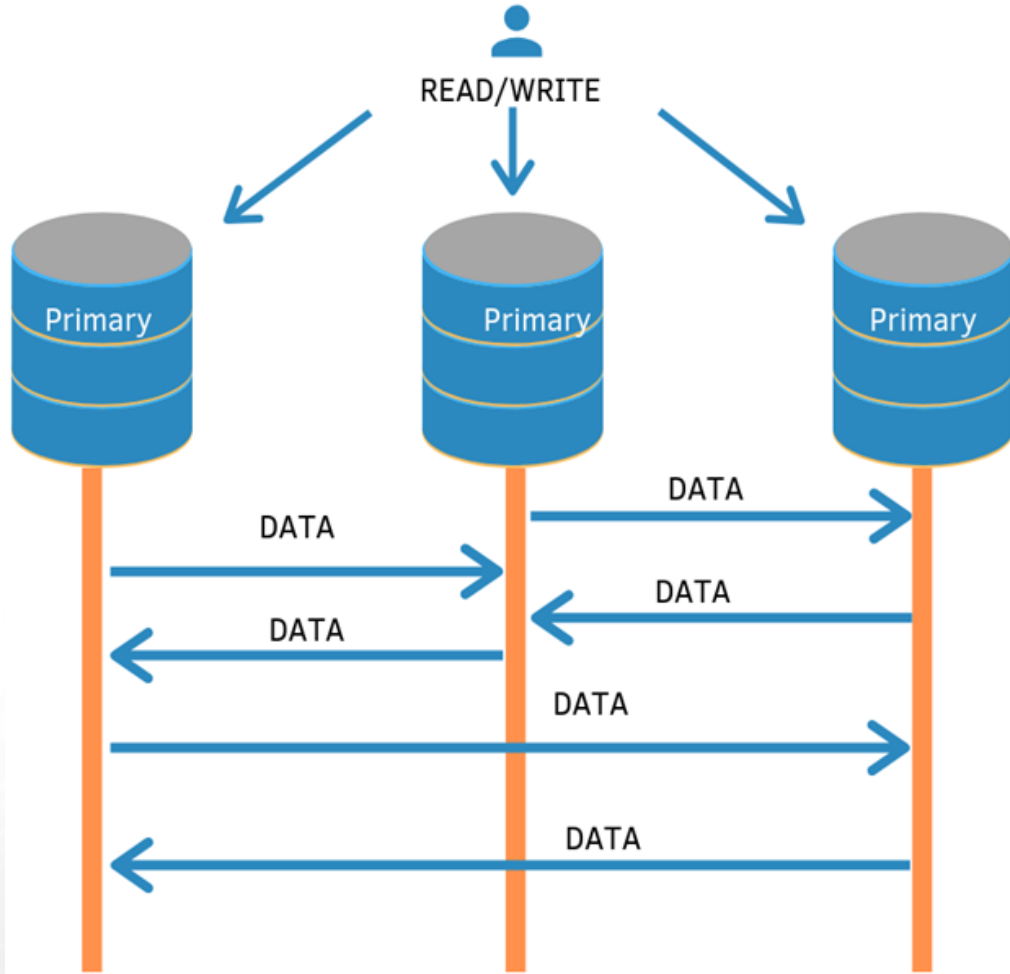
Данные между серверами автоматически синхронизируются, чтобы оставаться одинаковыми.

Достоинства Master-Master



- Высокая отказоустойчивость: при отказе одного сервера его функции автоматически переходят к другому, обеспечивая непрерывность работы
- Балансировка нагрузки: входящие запросы распределяются между серверами, что предотвращает перегрузку каждого отдельного узла и повышает общую производительность.
- Снижение задержек для распределённых пользователей: клиенты могут обращаться к географически ближайшему серверу, что уменьшает время отклика

Недостатки Master-Master

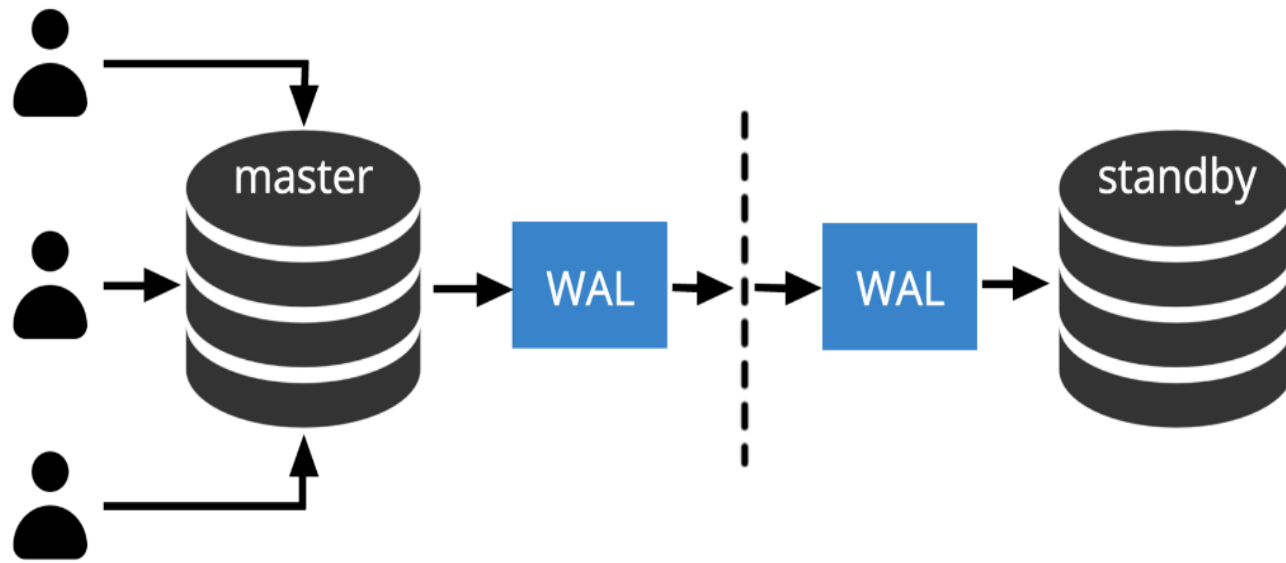


- При одновременном изменении одних и тех же данных на разных серверах возникает риск нарушения их согласованности.
- Синхронизация данных между серверами происходит не мгновенно. Это приводит к временным расхождениям, когда пользователи могут видеть устаревшие данные.
- Требуется реализация сложных механизмов для разрешения конфликтов и контроля синхронизации, что увеличивает операционные расходы.

Репликация в СУБД PostgreSQL

Тип	Описание	Преимущества
Физическая	Этот метод основан на копировании файлов на уровне блоков данных, что позволяет поддерживать полное соответствие между главным и подчинёнными серверами. Используется запись всех изменений в журнал WAL и их передача на реплики.	Высокая производительность, полная копия данных, минимальная задержка при передаче изменений.
Логическая	Логическая репликация предполагает передачу изменений на уровне отдельных таблиц или групп данных. Этот способ позволяет более гибко настраивать синхронизацию и выбирать только необходимые данные для репликации.	Гибкость настройки, возможность частичной синхронизации, отсутствие необходимости полной копии базы данных.

Физическая репликация



WAL (Write-Ahead Log) — это журнал предзаписи, который обеспечивает целостность данных, записывая все изменения в отдельный журнал перед их применением к основным файлам данных.

Физическая репликация

Потоковая репликация бывает двух видов — асинхронная и синхронная.

Асинхронная репликация: первоначально изменения применяются на основном узле и только потом отправляются записи из WAL на реплики.

Преимущество: быстрое подтверждение транзакции, т.к. не нужно ждать пока все реплики применят изменения.

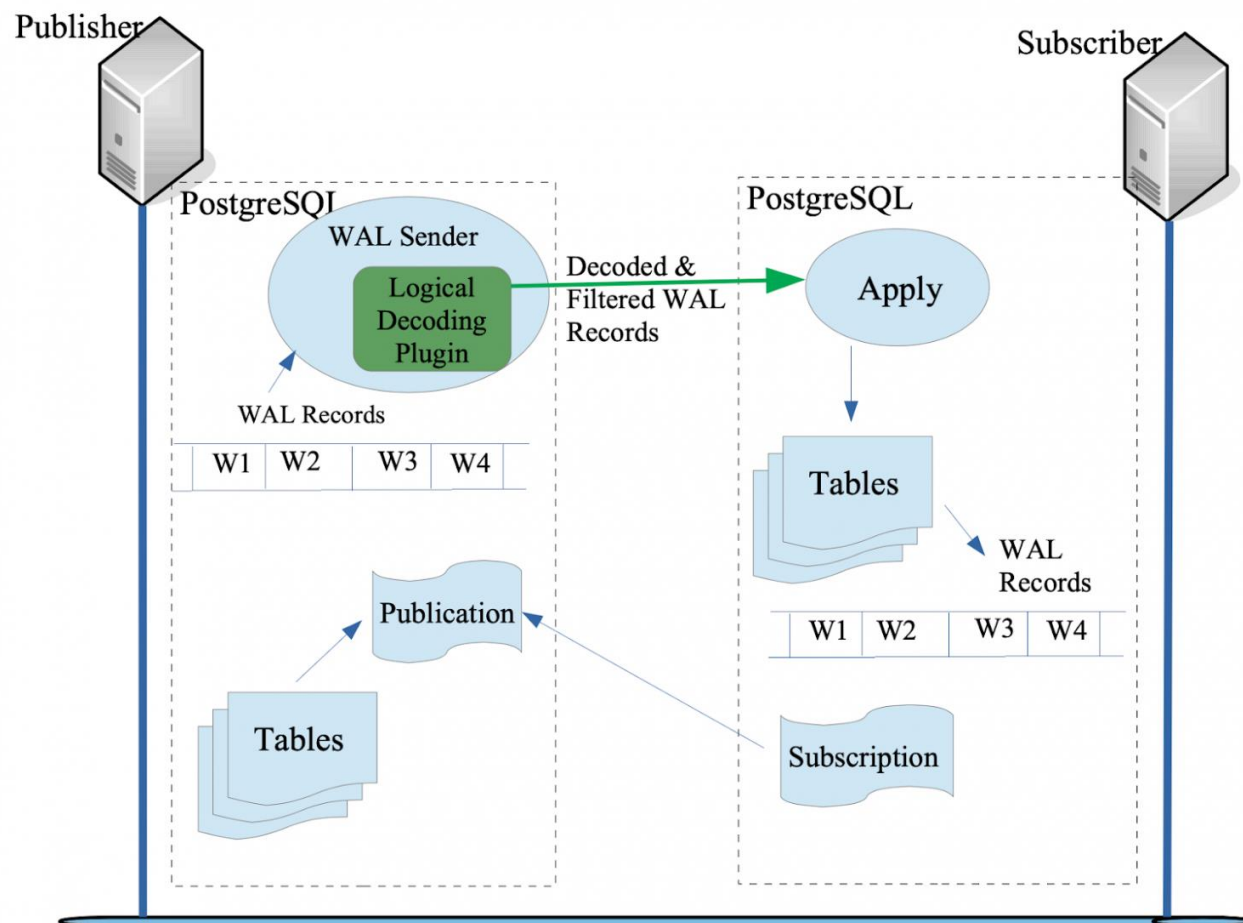
Недостаток: в том, что при падении основного сервера часть данных на репликах может потеряться, так как изменения не успели продублироваться.

Синхронная репликация: изменения сначала записываются в WAL хотя бы одной реплики и только после этого фиксируются на основном сервере.

Преимущество: более надежный способ, при котором сложнее потерять данные.

Недостаток: операции выполняются медленнее, потому что прежде чем подтвердить транзакцию, нужно сначала продублировать ее на реплике.

Логическая репликация

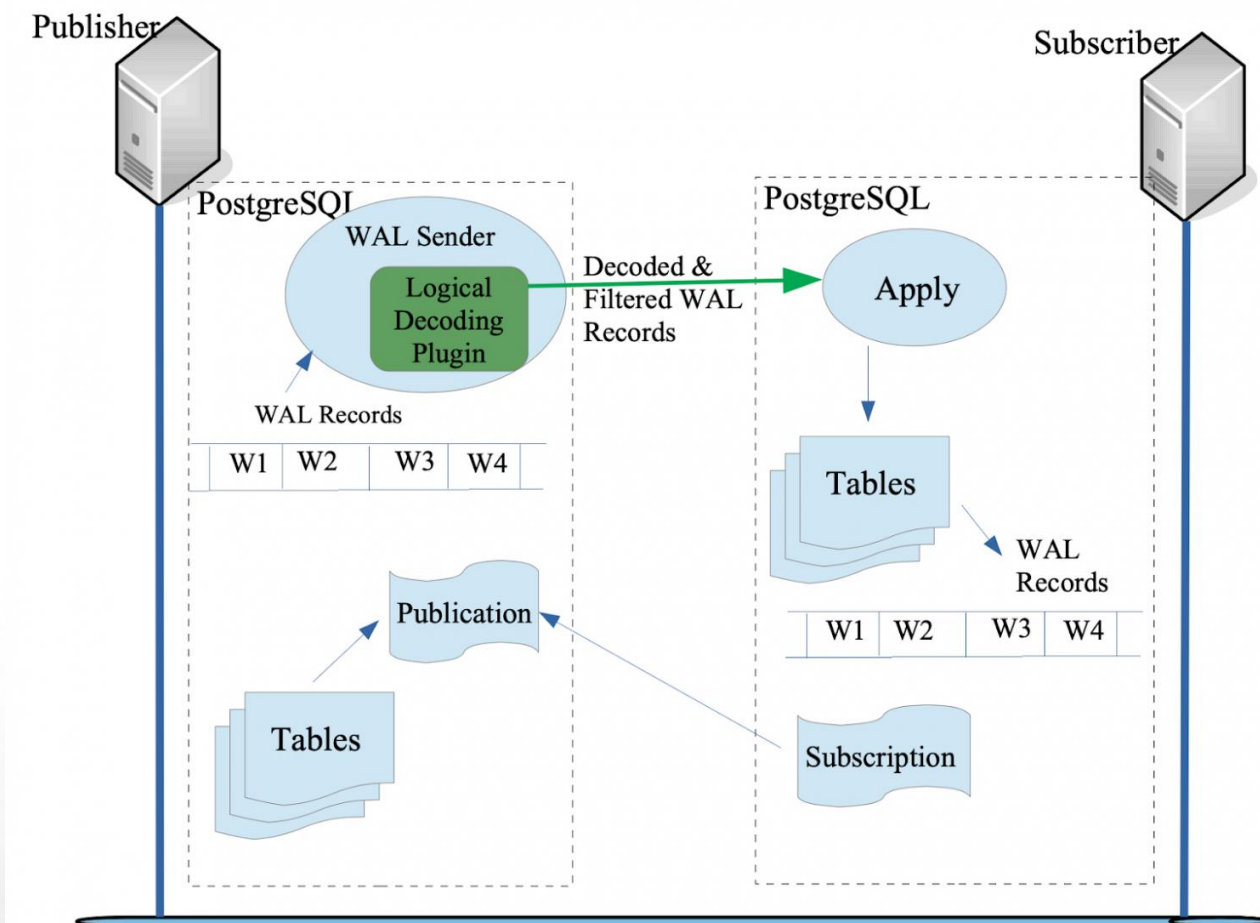


Логическая репликация — метод копирования данных из одной базы в другую, который передает не физические файлы, а сами изменения или отдельные таблицы, основываясь на их логических идентификаторах

Логическая репликация определяет два объекта: издатель (publisher) и подписчик (subscriber).

Издатель — это узел, который определяет определенную группу таблиц (называемую публикацией), на которую может подписаться подписчик, создав подписку для получения изменений в этой конкретной группе таблиц.

Логическая репликация



Публикация — это набор изменений, выделяемых в таблице или в группе таблиц.

Публикации могут ограничивать набор публикуемых изменений, выбирая любое сочетание операций DML.

У каждой публикации может быть множество подписчиков.

Подписчики - подписываются на одну или несколько публикаций на публикующем узле (издателе).

Подписчики получают данные из публикаций, на которые они подписаны, и могут затем повторно опубликовать данные для организации каскадной репликации

Логическая репликация

1. Создание публикации

```
CREATE PUBLICATION my_publication FOR TABLE users, orders;
```

2. Публикация всех таблиц

```
CREATE PUBLICATION all_tables FOR ALL TABLES;
```

Публикация с фильтрацией операций (реплицируются только данные, внесенные в публикацию командой Insert)

```
CREATE PUBLICATION insert_only FOR TABLE users WITH (publish = 'insert');
```

3. Создание подписки:

```
CREATE SUBSCRIPTION my_subscription CONNECTION 'host=192.168.1.100 port=5432  
dbname=mydb user=replicator' PUBLICATION my_publication;
```

Создание подписки с параметрами:

```
CREATE SUBSCRIPTION my_subscription CONNECTION 'host=192.168.1.100 port=5432  
dbname=mydb user=replicator' PUBLICATION pub1, pub2 WITH ( copy_data = true,  
create_slot = true, enabled = true );
```

Управление подписками

1. Приостановление/ Возобновление подписки

```
ALTER SUBSCRIPTION my_subscription DISABLE/ ENABLE;
```

2. Обновление соединения

```
ALTER SUBSCRIPTION my_subscription CONNECTION  
'host='new-server' port=5432 dbname=mydb  
user=replicator';
```

3. Удаление подписки

```
DROP SUBSCRIPTION my_subscription;
```

4. Создание пользователя для репликации

```
CREATE ROLE replicator WITH REPLICATION LOGIN  
PASSWORD 'secure_password';  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO  
replicator;
```

Для настройки логической репликации в файле `postgresql.conf` необходимо установить значения следующий параметров:

`wal_level = logical,`

`max_wal_senders = 10 (> 1),`

`max_replication_slots = 10 (>1)`

Пример логической репликации

На основном сервере (издателя):

1. Создать таблицы

```
CREATE TABLE t1(a int, b text, PRIMARY KEY(a));  
CREATE TABLE t2(c int, d text, PRIMARY KEY(c));  
CREATE TABLE t3(e int, f text, PRIMARY KEY(e));
```

2. Заполнить их данными

```
INSERT INTO t1 VALUES (1, 'one'), (2, 'two'), (3, 'three');  
INSERT INTO t2 VALUES (1, 'A'), (2, 'B'), (3, 'C');  
INSERT INTO t3 VALUES (1, 'i'), (2, 'ii'), (3, 'iii');
```

3. Создать подписки

```
CREATE PUBLICATION pub1 FOR TABLE t1;  
CREATE PUBLICATION pub2 FOR TABLE t2 WITH (publish = 'truncate');  
CREATE PUBLICATION pub3a FOR TABLE t3 WITH (publish = 'truncate');  
CREATE PUBLICATION pub3b FOR TABLE t3 WHERE (e > 5);
```

Пример логической репликации

На стороне подписчика:

1. Создать таблицы

```
CREATE TABLE t1(a int, b text, PRIMARY KEY(a));  
CREATE TABLE t2(c int, d text, PRIMARY KEY(c));  
CREATE TABLE t3(e int, f text, PRIMARY KEY(e));
```

2. Создать подписки

```
CREATE SUBSCRIPTION sub1 CONNECTION 'host=127.0.0.1 port=5432 dbname=shop  
user=postgres password=postgres' PUBLICATION pub1;  
CREATE SUBSCRIPTION sub2 CONNECTION 'host=127.0.0.1 port=5432 dbname=shop  
user=postgres password=postgres' PUBLICATION pub2;  
CREATE SUBSCRIPTION sub3 CONNECTION 'host=127.0.0.1 port=5432 dbname=shop  
user=postgres password=postgres' PUBLICATION pub3a, pub3b;
```


Пример логической репликации

На стороне подписчика:

1. Создать таблицы

```
CREATE TABLE t1(a int, b text, PRIMARY KEY(a));  
CREATE TABLE t2(c int, d text, PRIMARY KEY(c));  
CREATE TABLE t3(e int, f text, PRIMARY KEY(e));
```

2. Создать подписки

```
CREATE SUBSCRIPTION sub1 CONNECTION 'host=127.0.0.1 port=5432 dbname=shop  
user=postgres password=postgres' PUBLICATION pub1;  
CREATE SUBSCRIPTION sub2 CONNECTION 'host=127.0.0.1 port=5432 dbname=shop  
user=postgres password=postgres' PUBLICATION pub2;  
CREATE SUBSCRIPTION sub3 CONNECTION 'host=127.0.0.1 port=5432 dbname=shop  
user=postgres password=postgres' PUBLICATION pub3a, pub3b;
```

3. Данные синхронизировались: SELECT * FROM t1;

a [PK] integer	b text
1	one
2	two
3	three

Пример логической репликации

На стороне издателя:

4. Вставка дополнительных данных в таблицы на стороне публикации:

INSERT INTO t1 VALUES (4, 'four'), (5, 'five'), (6, 'six');

INSERT INTO t2 VALUES (4, 'D'), (5, 'E'), (6, 'F');

INSERT INTO t3 VALUES (4, 'iv'), (5, 'v'), (6, 'vi');

a [PK] integer	b text
1	one
2	two
3	three
4	four
5	five
6	six

c [PK] integer	d text
1	A
2	B
3	C
4	D
5	E
6	F

e [PK] integer	f text
1	i
2	ii
3	iii
4	iv
5	v
6	vi

Пример логической репликации

На стороне подписчика:

4. Выборка данных дала результат:

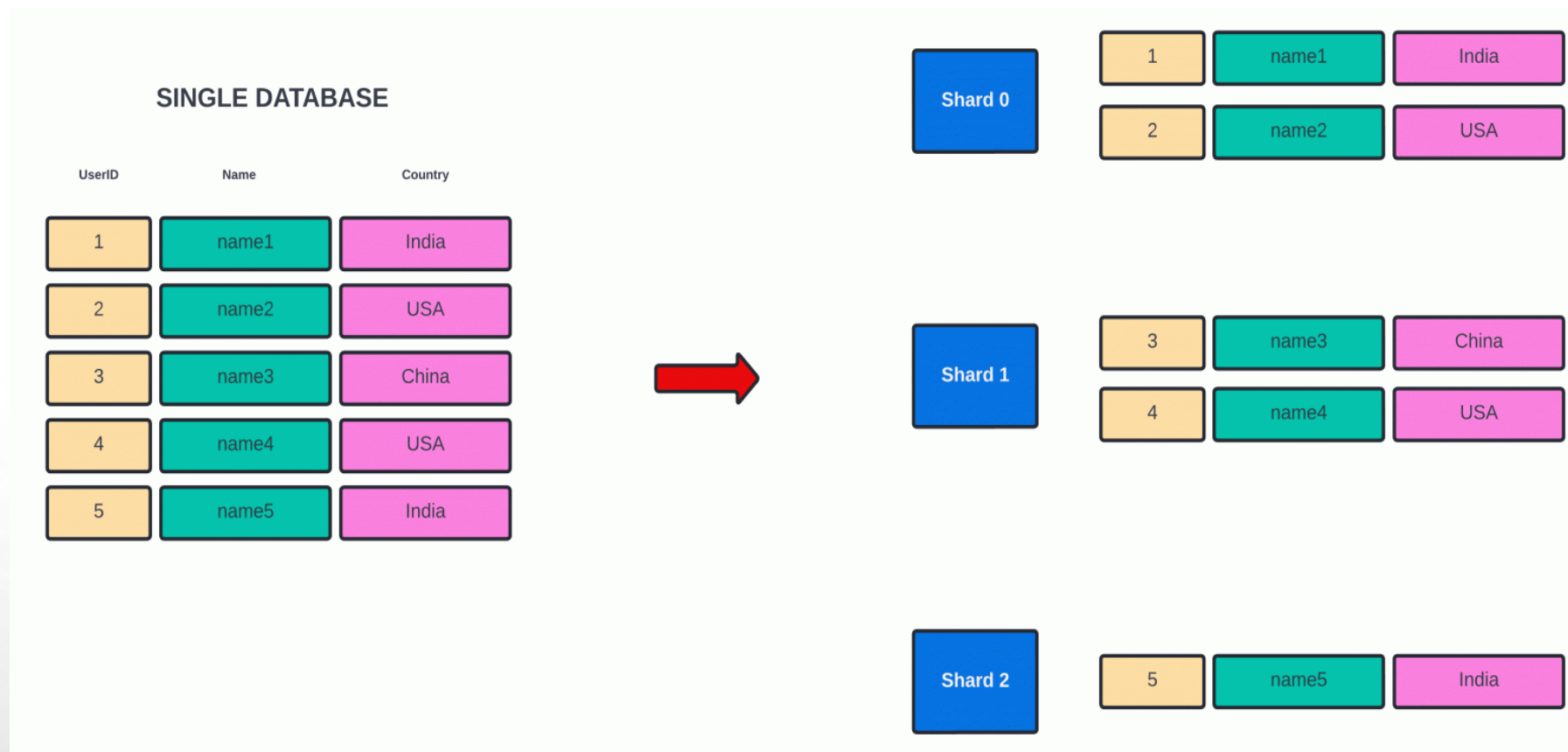
a [PK] integer	b text
1	one
2	two
3	three
4	four
5	five
6	six

c [PK] integer	d text
1	A
2	B
3	C

a [PK] integer	b text
1	one
2	two
3	three
6	six

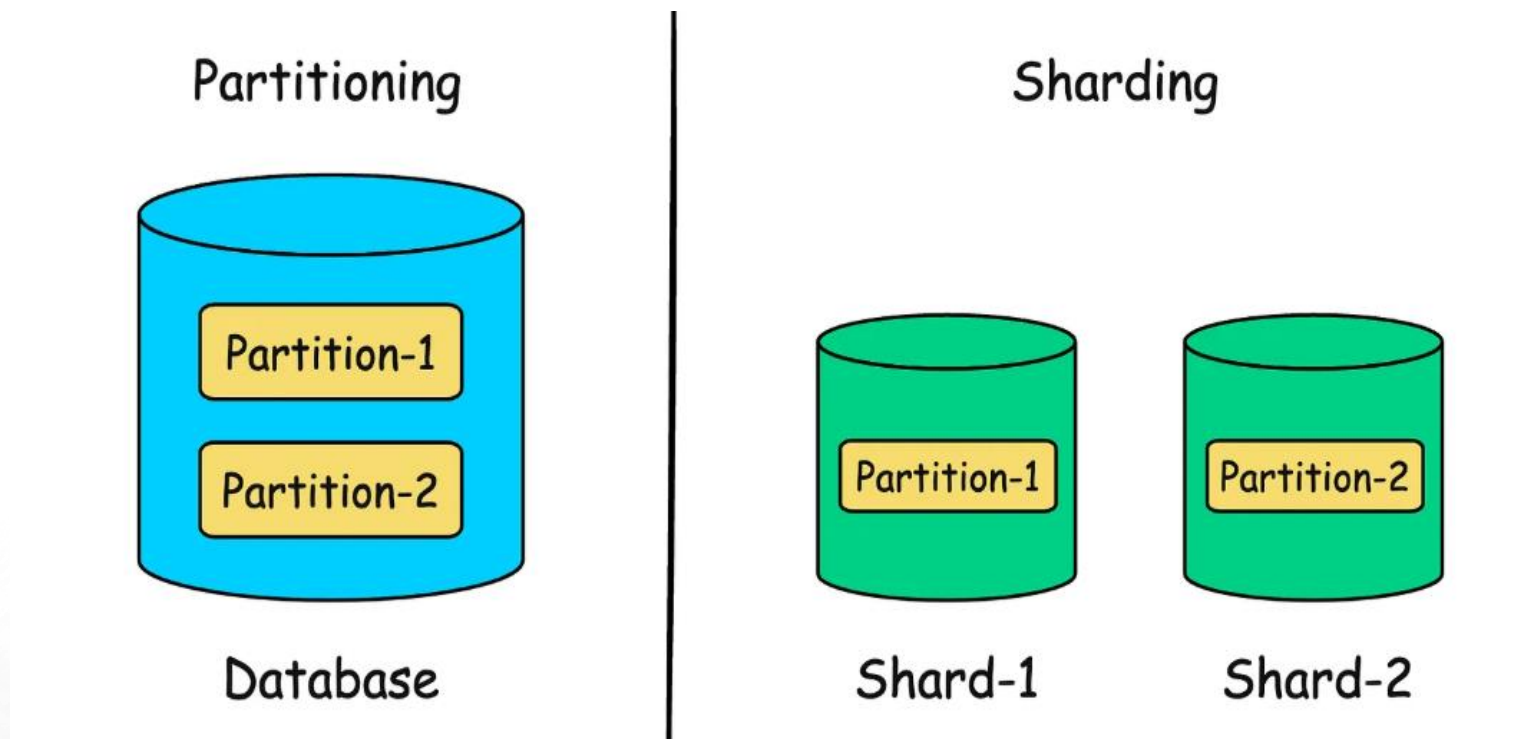
Шардирование баз данных

Шардирование — это метод горизонтального масштабирования базы данных, при котором данные разделяются на независимые части (шарды) и хранятся на разных серверах.



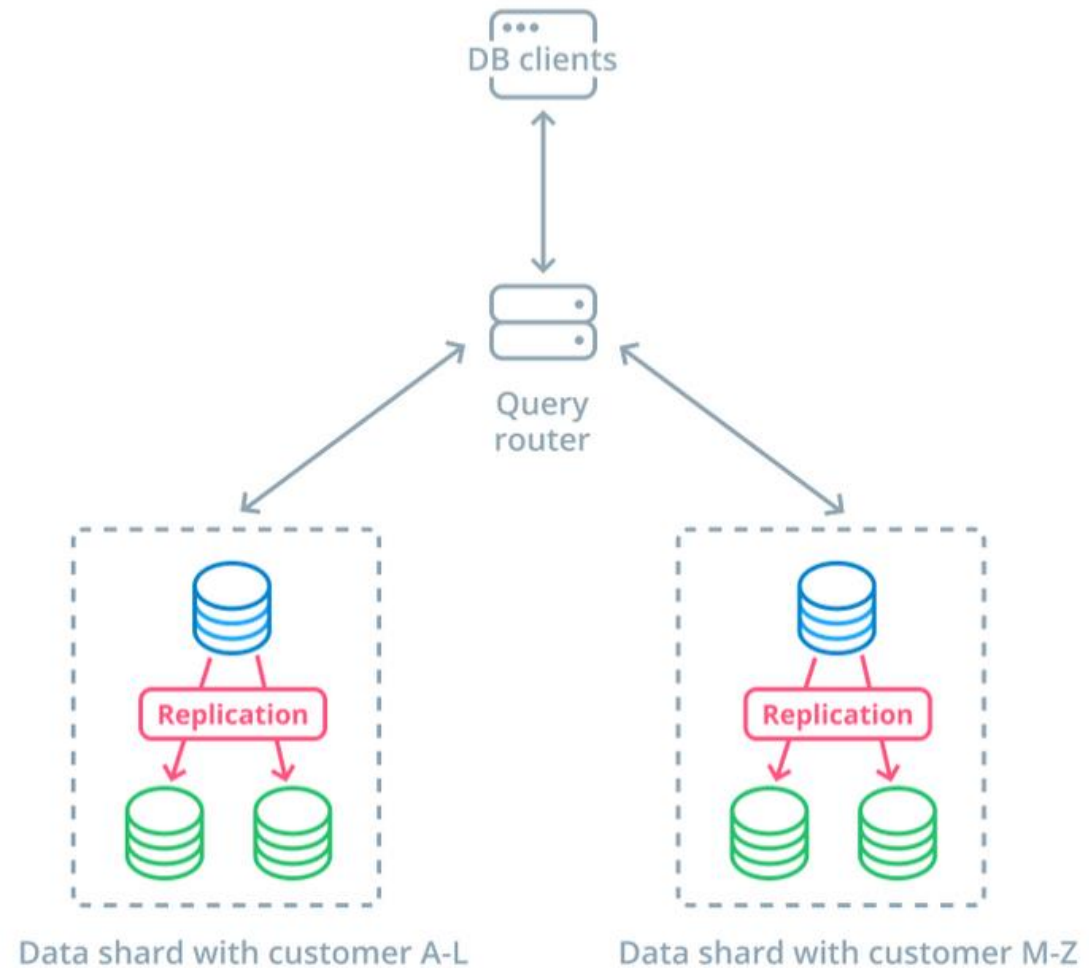
Шардирование vs Партицирование

Партиционирование делит таблицы внутри одной СУБД, шардирование - распределяет данные по нескольким экземплярам СУБД.



Архитектура шардированной системы

- Клиентское приложение выполняет обычные SQL-запросы, не зная о наличии нескольких шардов.
- Маршрутизатор перехватывает запрос, по значению ключа шардинга определяет, к какому серверу (шарду) обратиться, и перенаправляет запрос туда.
- Шард выполняет полученный запрос и возвращает результат обратно маршрутизатору, который уже отдает данные клиенту.
- Каждый шард в общем случае представляет собой полноценную базу PostgreSQL, которая может иметь репликацию для отказоустойчивости. Набор реплик, относящихся к одному шару **реплика-сетом**.



Алгоритмы шардирования

В основе шардирования лежит **алгоритм распределения записей между шардами**. Его ключевые задачи:

- Однозначно определить, на какой шард должна быть записана новая строка данных.
- По заданному ключу шардинга быстро находить шард, на котором хранится искомая запись для операций чтения.

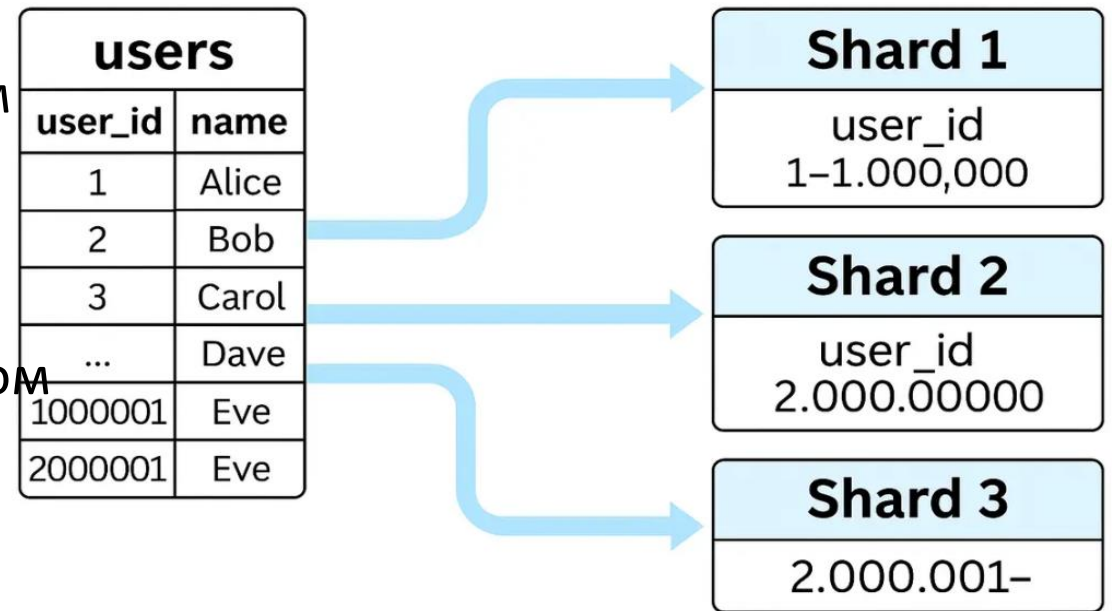
1. Range-based Sharding

В этом методе данные распределяются по шардам на основе определённых диапазонов значений.

+: Легкость реализации
Запросы эффективны, если ограничены диапазоном

-: Возможен дисбаланс нагрузки, если некоторые диапазоны используются чаще

Трудности при изменении границ диапазонов



Алгоритмы шардирования

2. Key-based Sharding (Хеширование)

В этом методе для распределения данных используется хеш-функция.

- выбираем поле таблицы, как ключ шардирования;
- хеш-функция вычисляется его хеш;
- полученное число отображается на индекс шарда – берется остаток от деления хеша на количество шардов.



$$548291 \% 3 = 2$$

+ : Данные равномерно

распределяются между шардами

-: Трудно добавить новые

шарды без перераспределения данных

Алгоритмы шардирования

3. Directory-Based Sharding (Каталожное)

Directory-based подход вводит отдельный каталог – централизованную таблицу маршрутизации, где хранится отображение диапазонов или конкретных значений ключа на идентификаторы шардов:

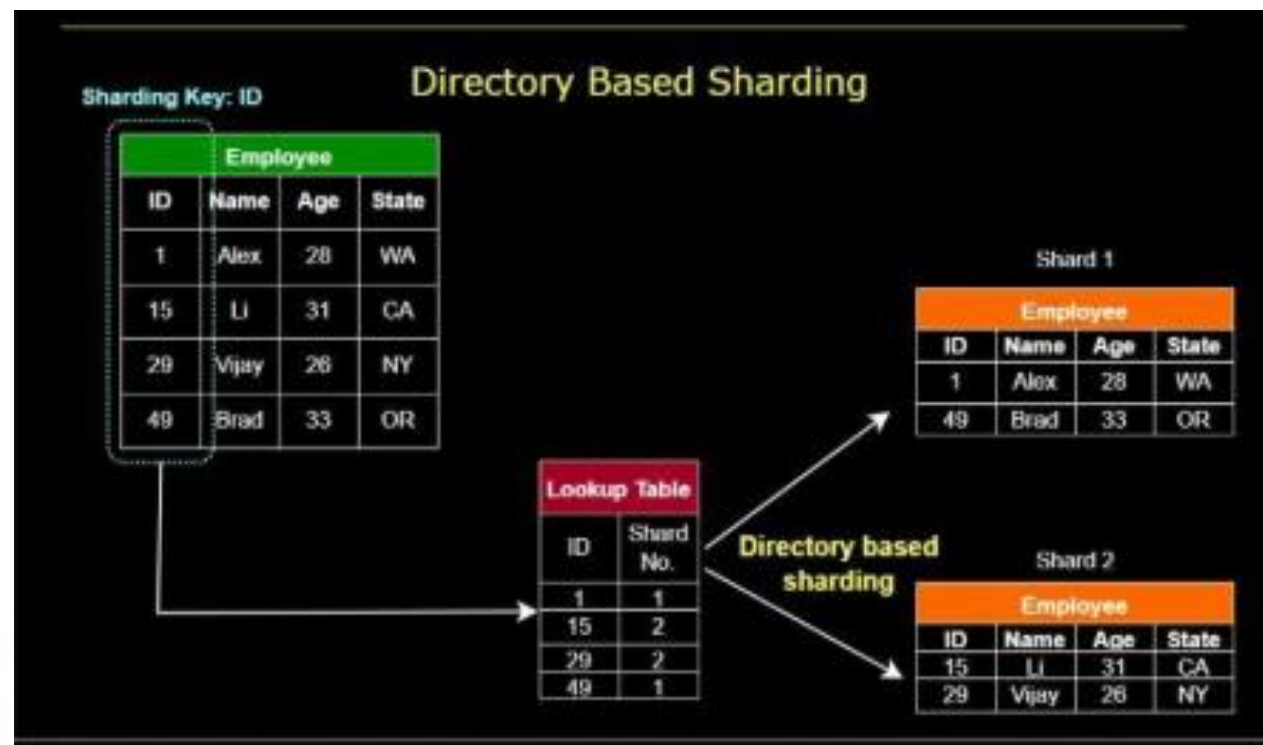
- маршрутизатор сначала делает запрос в каталог соответствий, получает ответ;
- направляет основной запрос на нужный узел.

+:

- Гибкость в распределении данных
- Можно менять логику шардирования без перераспределения всей базы

-:

- Каталог является единой точкой отказа
- Дополнительные задержки при доступе к данным



Подходы к шардированию

- **Ручное шардирование** и логика на уровне приложения – когда всю ответственность за разбиение базы берет на себя разработчик и код приложения.
- **Использование расширений распределенной БД** – например, Citus и др., которые добавляют в PostgreSQL прозрачный уровень шардирования
- **Шардирование через Foreign Data Wrapper (FDW)** – объединение нескольких серверов PostgreSQL “вручную” средствами внешних таблиц и встроенного партиционирования.
- **Отдельные решения – специализированные кластеры PostgreSQL и надстройки** (Postgres-XL, Postgres-XC, Greenplum, Shardman и т.д.), а также распределенные NewSQL-СУБД, совместимые с протоколом Postgres.

Ручное шардирование

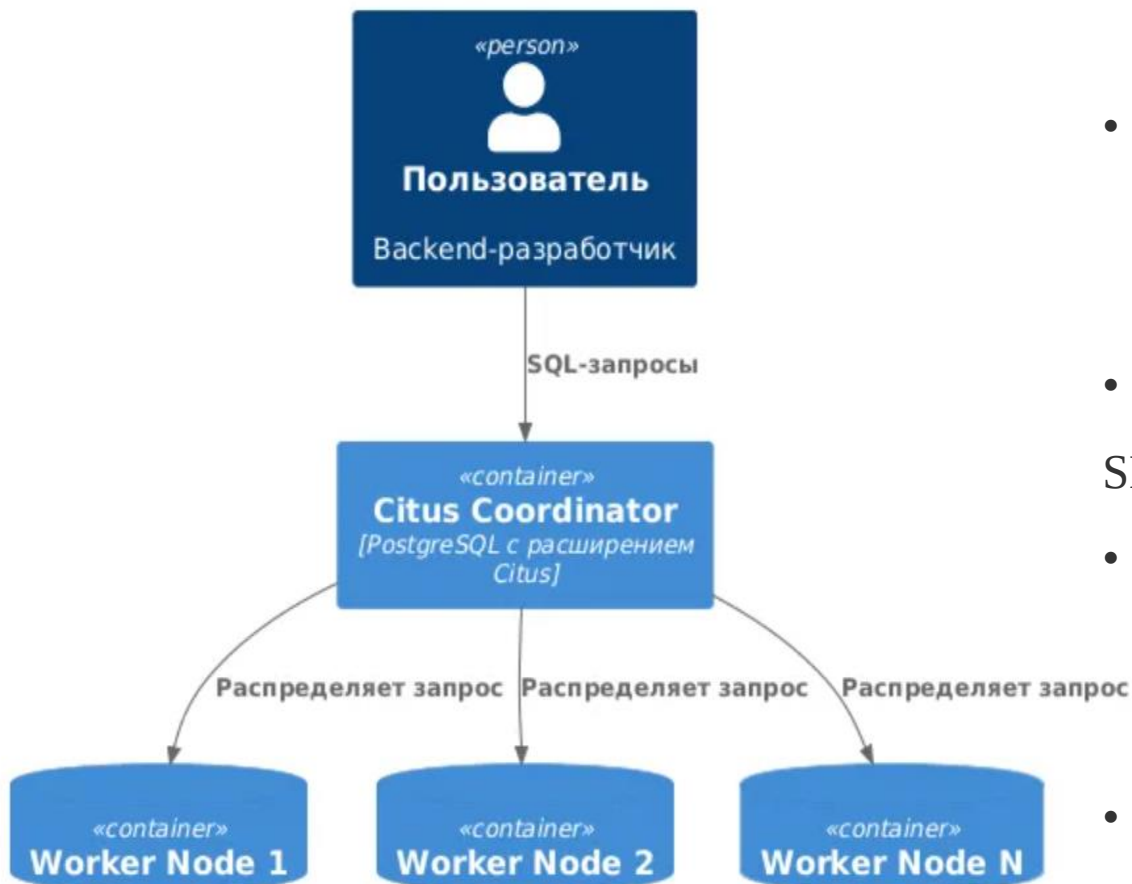


- Полный контроль
- Простота архитектуры – отсутствуют дополнительные слои.
- Нет зависимости от сторонних расширений
- Гибкость в выборе стратегии



- Сложность разработки и поддержки.
- Отсутствие кросс-шардовых возможностей по умолчанию.
- Нет поддержки распределенных транзакций.

Использование расширения Citus



- Один из узлов кластера обозначается координатором (master), остальные – рабочие узлы (workers).
- На координаторе устанавливается расширение citus (`CREATE EXTENSION citus;`), после чего он становится точкой входа для запросов.
- Регистрируются рабочие узлы, например:
`SELECT citus_add_node( 'worker2.citus.example.com', 5432);`
- Данные размещаются на шардах (партициях) рабочих узлов, а координатор хранит метаданные (внутренние таблицы с распределением шардов).
- Когда приходит запрос от приложения на координатор, Citus определяет, к каким шардам его направить, запускает параллельные запросы на воркерах и агрегирует результат.

Использование расширения Citus

Алгоритм работы:

1. На координаторе создается таблица

```
CREATE EXTENSION citus;
```

```
CREATE TABLE app_logs (
```

```
  log_id bigserial,
```

```
  service_name text NOT NULL,
```

```
  user_id int,
```

```
  log_time timestamptz default now(),
```

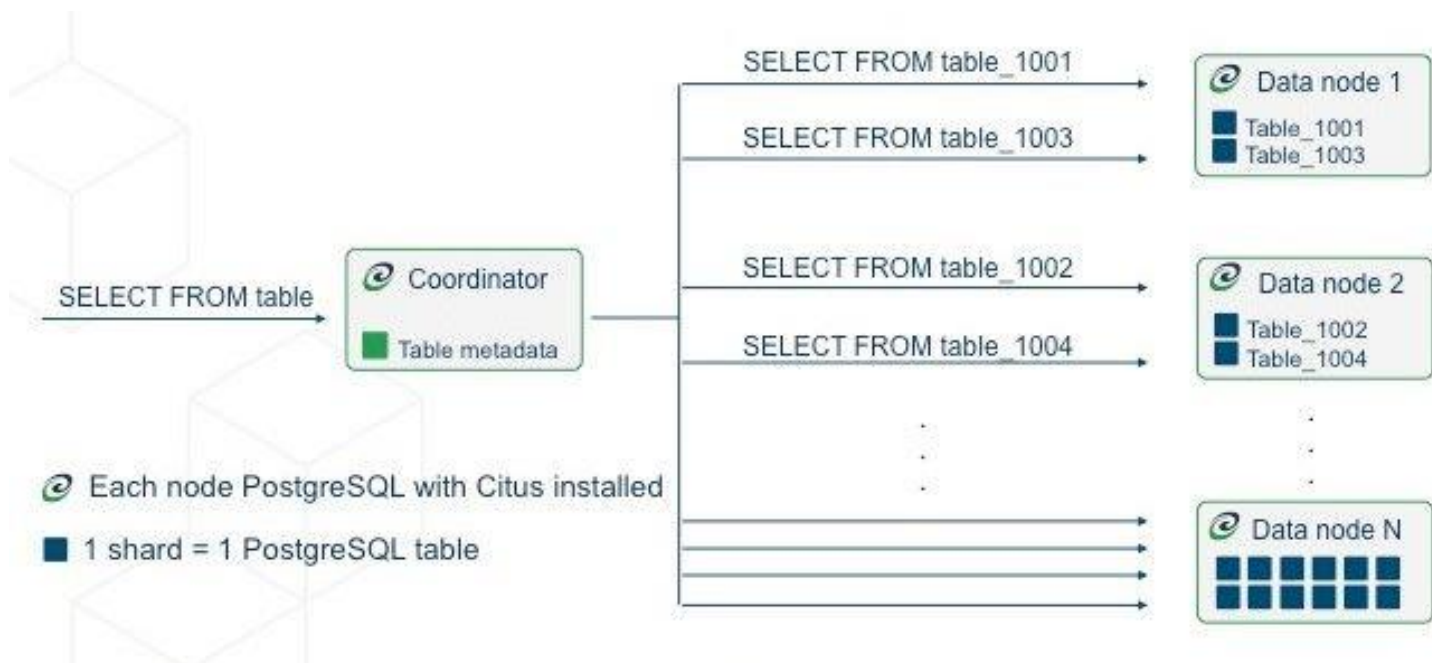
```
  message text,
```

```
  PRIMARY KEY (log_id, service_name)
```

```
);
```

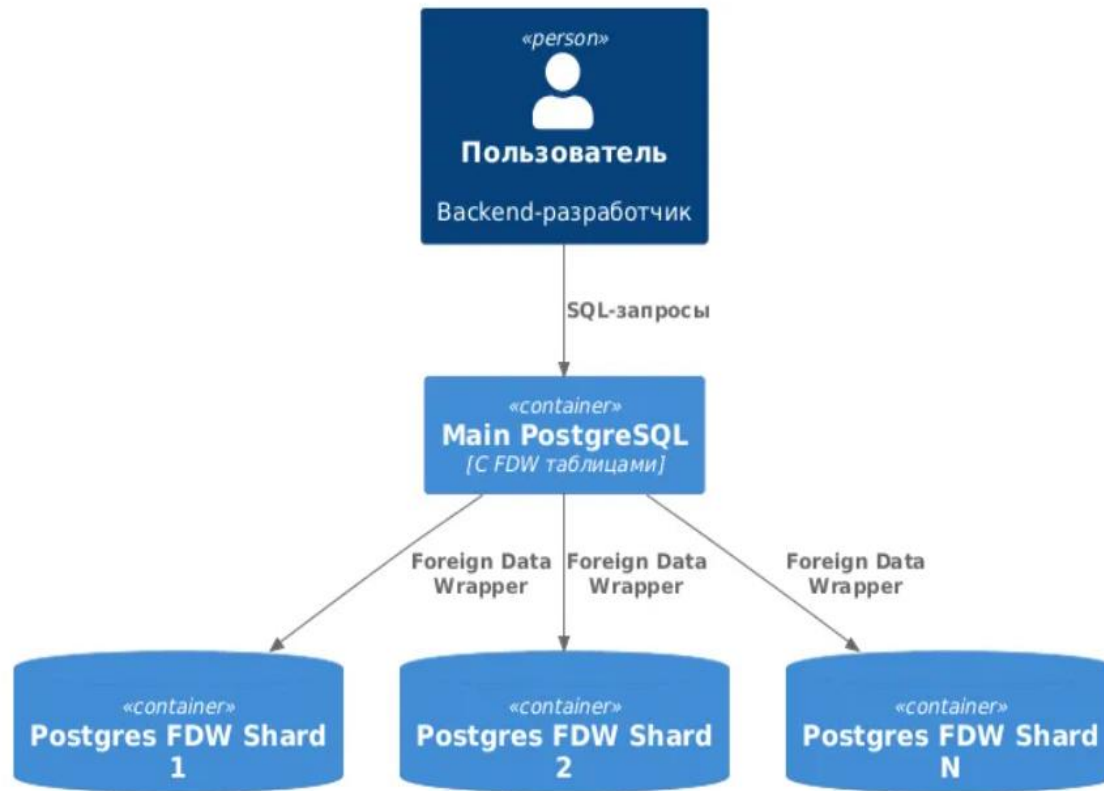
2. Логическая таблица разделяется на шарды, допустим по значению service_name (по умолчанию используя хеш-шардирование) и распределяется по рабочим узлам.

```
SELECT create_distributed_table('app_logs',  
'service_name');
```



3. При манипулировании данными вычисляется хеш от service_name и отправляется запрос на соответствующий шард при наличии фильтрации, при отсутствии фильтрации – запрос рассылается параллельно на все узлы.

Использование Foreign Data Wrapper



- На головном координаторе создается партиционированная таблица. Критерием разделения выбирается ключ шардирования, например, `HASH(user_id)`.
- Для каждой партии, определенной в логической схеме, создается внешняя таблица, реализованная с помощью расширения `postgres_fdw`. Она представляет собой прокси-интерфейс к реальной физической таблице, расположенной на одном из удаленных серверов-шардов.
- Когда приложение выполняет SQL-запрос к логической партиционированной таблице на координаторе, планировщик проводит процедуру исключения ограничений: если предикат запроса позволяет однозначно определить целевую партицию, то планировщик исключает из плана выполнения все остальные партиции.

Использование Foreign Data Wrapper



- Используются только стандартные возможности PostgreSQL.
- Для клиентских приложений схема данных представляется единой таблицей.
- Шарды остаются независимыми серверами PostgreSQL, что упрощает их администрирование и резервное копирование.



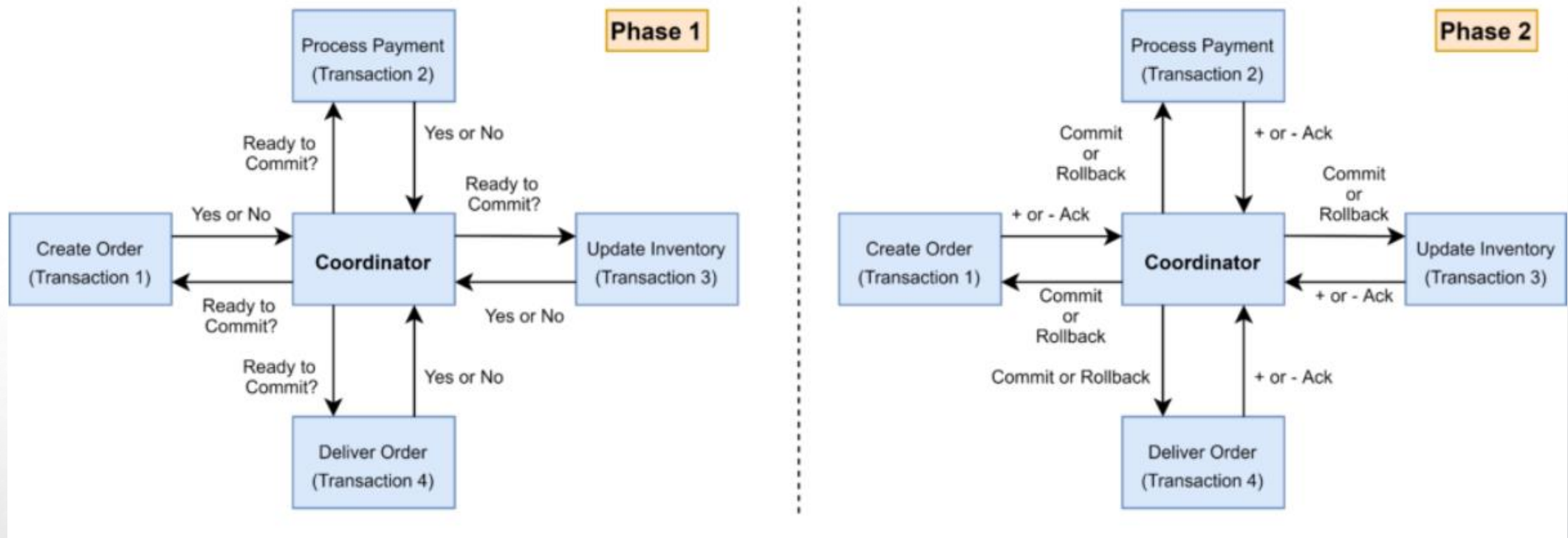
- Запросы, затрагивающие несколько шардов, требуют пересылки всех данных на координатор, что может создавать узкое место.
- Механизмы распределенных транзакций в `postgres_fdw` имеют ограничения, что усложняет поддержание строгой ACID-семантики при записи в несколько шардов одновременно.
- Изменение схемы данных может требовать синхронного выполнения на всех шардах.
- Отсутствие встроенной репликации между шардами

Проблемы, возникающие при использовании шардирования

1. Кросс-шардовые запросы

1.1. Использовать принцип конечной согласованности (eventual consistency)

1.2. Двухфазный протокол фиксации транзакций (2PC)

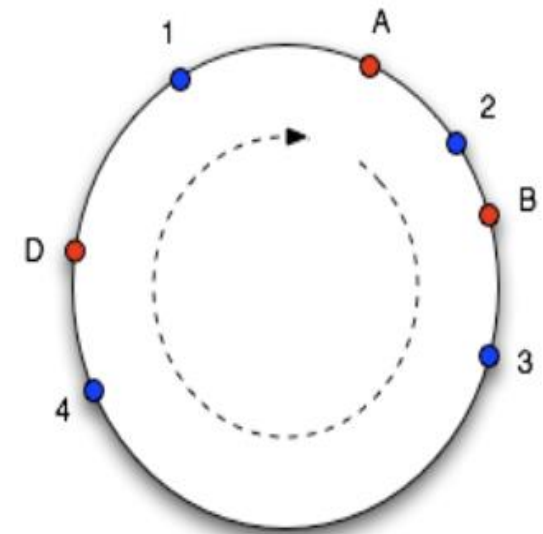
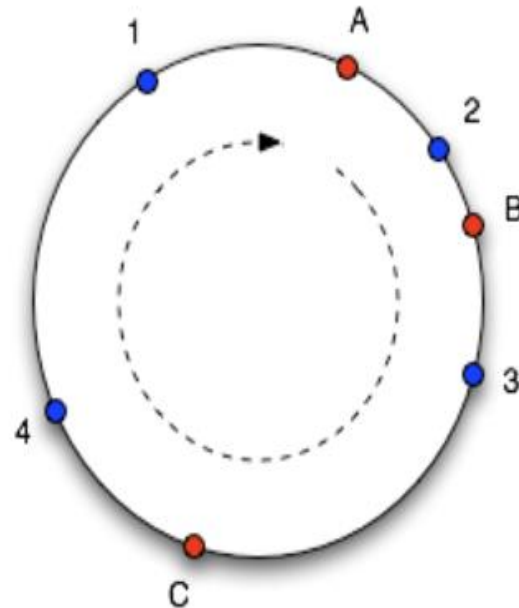
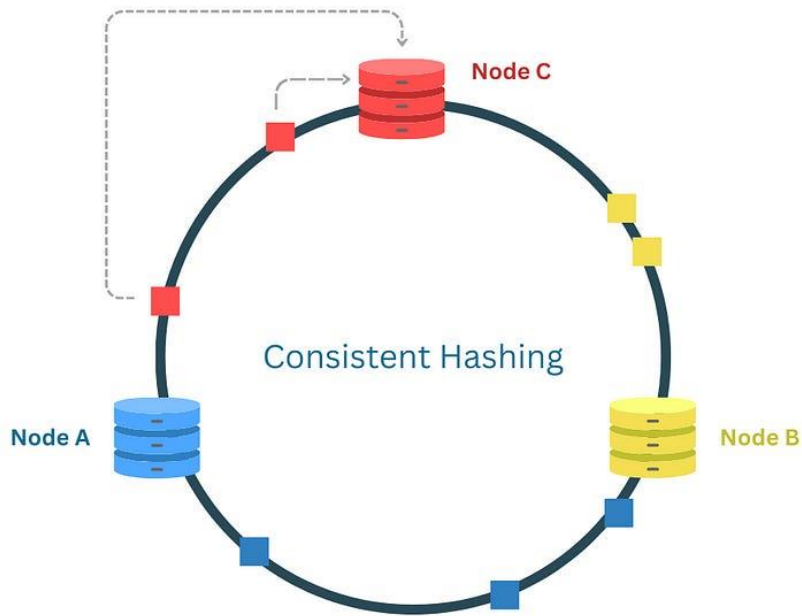


Проблемы, возникающие при использовании шардирования

2. Перегруженный шард (Hotspot)

3. Решардинг - процесс перераспределения данных при добавлении или удалении серверов

3.1. Консистентное хеширование



Спасибо за внимание!